PEMBANGUNAN DATA WAREHOUSE AKADEMIK UNTUK MENDUKUNG ANALISIS DAN PENGAMBILAN KEPUTUSAN DI BIRO AKADEMIK

Tugas Besar Pergudangan Data

Nama Anggota Kelompok 18:

Keren Marito	(123450020)
Aditya Taufiqurrohman	(123450032)
Lia Hana Ichisasmita	(123450089)

PROGRAM STUDI SAINS DATA
FAKULTAS SAINS
INSTITUT TEKNOLOGI SUMATERA

2025

DAFTAR ISI

DAFTAR ISI **2**
1. Project Overview 4
2. Key Achievements 4
3. Business Impact 5
4. Recommendations 5
BAB 1 PENDAHULUAN **6**
1.1 Latar Belakang 6
1.2 Tujuan 6
1.3 Ruang Lingkup 7
1.4 Metodologi 7
BAB 2 REQUIREMENTS ANALYSIS **8**
2.1 Business Requirements (Kebutuhan Bisnis) 8
2.2 Identifikasi Stakeholder 8
2.3 Proses Bisnis 9
2.4 Functional Requirements (Kebutuhan Fungsional) 9
2.5 Non-Functional Requirements (Kebutuhan Non-Fungsional) 10
2.6 Business Rules (Aturan Bisnis) 10
2.7 Data Sources (Sumber Data) 10
2.7.1 Identifikasi Sumber Data 10
2.7.2 Analisis Sumber Data 11
2.7.3 Data Profiling 11
BAB 3 PERANCANGAN SISTEM **12**
3.1 Conceptual model (ERD) 12
3.2 Logical model (dimensional model) 13
3.2.1 Fact Table: Fakta_KinerjaAkademik 13
3.2.2 Dimension Tables 14
3.3 Physical design 14
3.3.1 Database Configuration 14
3.3.2 Table Partitioning 15
3.3.3 Indexing Strategy 15
3.4 System Architecture 15
BAB 4 IMPLEMENTASI **16**
4.1 Database Implementation 16
4.1.1 Schema Creation 16
4.1.2 Partitioning & Indexing 17
4.2 ETL Process Implementation 18
4.2.1 Control Flow ETL 19
4.2.2 Data Flow Logic 20
4.3 Dashboard Development 20

4.3.1 Data Modeling....21
4.3.2 Dashboard Layout....21
4.4 Security Implementation.... 23
BAB 5 PENGUJIAN & VALIDASI....25
5.1 Data Quality Assurance (DQA)....25
5.2 Performance Testing....27
5.2.1 Skenario 1 — Query Analitik Menggunakan Index.... 27
5.2.2 Skenario 2 — Query Berbasis Tanggal (Partition Simulation).... 27
5.3 User Acceptance Test (UAT)....28
5.4 Bug Fixes....29
BAB 6 DEPLOYMENT & OPERATIONS....30
6.1 Production Deployment.... 30
6.1.1 Environment Specification.... 31
6.1.2 Deployment Steps....31
6.2 Backup Strategy.... 32
6.2.1 Jenis Backup....32
6.2.2 Lokasi Penyimpanan....32
6.2.3 Recovery Procedure.... 33
6.3 Monitoring....33
6.3.1 ETL Monitoring....33
6.3.2 Performance Monitoring....33
6.3.3 Data Freshness Monitoring....33
6.3.4 Power BI Monitoring....33
6.4 Maintenance Procedures....33
6.4.1 Database Maintenance....33
6.4.2 ETL Maintenance....34
6.4.3 Dashboard Maintenance....34
6.4.4 Security & Access Control.... 34
BAB 7 HASIL DAN PEMBAHASAN....34
7.1 Temuan Kunci Bisnis.... 34
7.2 Technical Performance Metrics....35
7.3 Challenges & Solutions....36
BAB 8 PENUTUP....37
8.1 Kesimpulan....37
8.2 Saran Pengembangan....38
DAFTAR PUSTAKA.... 39
LAMPIRAN....40

EXECUTIVE SUMMARY

1. Project Overview

Proyek ini bertujuan membangun Data Warehouse (DW) Akademik untuk menyediakan sumber data terintegrasi yang mendukung pelaporan, analisis, dan pengambilan keputusan di Biro Akademik. DW dirancang menggunakan dimensional model (star schema) dengan tabel dimensi, tabel fakta, serta proses ETL (Extract, Transform, Load) yang terstruktur. Dataset mahasiswa diolah melalui staging lalu dimuat ke DW untuk menghasilkan data yang bersih, konsisten, historis, dan siap dianalisis. Proyek mencakup perancangan model, pembuatan database, implementasi ETL, data quality check, optimasi performa query, security berbasis role, dan penjadwalan ETL otomatis.

2. Key Achievements

a. **Perumusan Kebutuhan Bisnis**

   Tim berhasil mengidentifikasi kebutuhan inti Biro Akademik, meliputi:

   - pengelolaan data mahasiswa (identitas, prodi, status),
   - monitoring nilai, IPK, dan progres studi,
   - pelaporan agregat berdasarkan fakultas, prodi, dan angkatan,
   - identifikasi mahasiswa berisiko (probation).

b. **Perancangan Star Schema**

   DW dibangun dengan model dimensional yang terdiri dari: Dim_Student (SCD Type 2), Dim_ProgramStudy, Dim_Status, Dim_Gender, Dim_Date, serta Fact_Enrollment. Model ini memungkinkan analisis multidimensi seperti tren waktu, performa prodi, dan status akademik.

c. **Implementasi ETL Otomatis**

   Tim mengembangkan proses ETL lengkap: validasi data di staging, pemrosesan dimensi secara incremental, pembuatan catatan historis pada Dim_Student, dan pemuatan data transaksi ke fact table. Seluruh proses dijalankan melalui stored procedure dan dikontrol oleh usp_Master_ETL.

d. **Data Quality Framework**

   Pemeriksaan kualitas data mencakup completeness, accuracy, referential integrity, duplicate checking, dan reconciliation antara staging dan warehouse, sehingga DW memiliki integritas yang baik.

e. **Optimasi Kinerja**

   DW dilengkapi dengan indeks (nonclustered, covering, columnstore), partisi tahun akademik, dan optimasi filtering serta join untuk meningkatkan performa query analitik.

f. **Analytical Views & Security**

Dua view analitik dibuat, yaitu vw_Student_Performance dan vw_Program_Analytics.

- Security menggunakan role-based access (executive, analyst, viewer, operator).
- ETL dijadwalkan otomatis setiap hari pukul 02.00.

3. Business Impact

Implementasi DW memberikan manfaat strategis, antara lain:

1. Efisiensi pengolahan data melalui integrasi yang mengurangi pekerjaan manual dan kesalahan rekap.
2. Dukungan keputusan akademik, seperti analisis IPK, performa prodi, revenue, dan tren semester.
3. Early Warning System, memungkinkan identifikasi mahasiswa berisiko secara lebih cepat.
4. Konsistensi laporan, karena semua pengguna mengakses satu sumber data yang sama.

4. Recommendations

Pengembangan lanjutan yang direkomendasikan adalah:

1. Integrasi dengan dashboard BI untuk visualisasi KPI real-time.
2. Penambahan modul data (KRS/KHS, kehadiran, data dosen, UKT).
3. Penerapan automated data quality alerts.
4. SCD Type 2 pada Dim Prodi dan Dim Status untuk historisasi lebih lengkap.
5. Pertimbangan migrasi ke cloud untuk skalabilitas.

BAB 1
PENDAHULUAN

1.1 Latar Belakang

Biro Akademik membutuhkan data yang terintegrasi, akurat, dan mudah dianalisis untuk mendukung proses pelaporan dan pengambilan keputusan. Namun, data mahasiswa yang digunakan sering tersebar, tidak historis, serta memerlukan proses rekap manual yang memakan waktu. Kondisi ini menyebabkan keterlambatan dalam evaluasi akademik, analisis performa mahasiswa, serta pemantauan program studi.

Untuk mengatasi masalah tersebut, dibangunlah Data Warehouse Akademik yang dirancang menggunakan dimensional model (star schema) agar data dapat disimpan secara terstruktur dan historis. Proses ETL (Extract, Transform, Load) diterapkan untuk melakukan pembersihan, transformasi, serta pemuatan data ke tabel dimensi dan fakta. Selain itu, dilakukan data quality checks, indexing, serta job scheduling agar data terjaga kualitasnya dan proses pemuatan berjalan otomatis.

Dengan adanya Data Warehouse ini, Biro Akademik dapat memperoleh sumber data terpusat yang mendukung analisis multidimensi seperti IPK mahasiswa, status akademik, tren nilai, dan performa program studi.

1.2 Tujuan

Pembangunan Data Warehouse Akademik ini bertujuan untuk:

1. Mengintegrasikan data mahasiswa dan informasi akademik ke dalam satu sistem yang konsisten dan terpusat.
2. Menyediakan data historis yang dapat ditelusuri menggunakan penerapan SCD Type 2 pada Dim_Student.
3. Mendukung analisis akademik seperti performa mahasiswa, tren semester, dan perkembangan program studi.
4. Mengotomasi proses ETL sehingga pemuatan data berjalan rutin tanpa intervensi manual.
5. Menjamin kualitas data melalui penerapan data quality checks, validasi, dan pemeriksaan integritas.

1.3 Ruang Lingkup

Ruang lingkup proyek mencakup:

1. Analisis kebutuhan bisnis dan data akademik.
2. Perancangan star schema terdiri dari Dim_Student, Dim_ProgramStudy, Dim_Status, Dim_Gender, Dim_Date, dan Fact_Enrollment.
3. Pembuatan database, tabel dimensi, tabel fakta, dan staging area.
4. Implementasi ETL untuk memuat data dari staging ke DW.
5. Penerapan data quality checks dan optimasi performa (index, columnstore, partition).
6. Pembuatan analytical views serta konfigurasi role-based security dan job scheduling.

1.4 Metodologi

Metodologi pembangunan Data Warehouse meliputi tahapan berikut:

1. Analisis Kebutuhan – Mengidentifikasi kebutuhan informasi dan KPI akademik.
2. Data Profiling – Menganalisis dataset mahasiswa untuk melihat struktur dan kualitas data.
3. Perancangan Dimensional Model – Mendesain star schema dan struktur historisasi data.
4. Pengembangan ETL – Melakukan extract, transform, load dengan stored procedure dan master ETL.
5. Data Quality & Optimasi – Melakukan validasi data serta menerapkan index dan tuning performa.
6. Security & Scheduling – Mengatur akses berbasis peran dan menjadwalkan ETL harian.

BAB 2
REQUIREMENTS ANALYSIS

2.1 Business Requirements (Kebutuhan Bisnis)

Kebutuhan bisnis pada sistem pengelolaan data akademik mahasiswa berfokus pada kebutuhan Biro Akademik untuk mengelola identitas mahasiswa, status studi, serta kinerja akademik secara terstruktur, akurat, dan siap dianalisis. Dataset yang digunakan memuat informasi dasar mahasiswa seperti NIM, Nama, Program Studi, Fakultas, Angkatan, Jenis Kelamin, Status Mahasiswa, serta kinerja akademik berupa Nilai, IPK, dan Semester.

Data tersebut merepresentasikan informasi yang secara nyata dikelola Biro Akademik untuk kebutuhan:

- Administrasi dan pemutakhiran data mahasiswa
- Pemantauan perkembangan akademik setiap semester
- Identifikasi mahasiswa dengan risiko akademik (probation)
- Penyusunan laporan fakultas dan program studi
- Rekapitulasi dan statistik mahasiswa aktif, cuti, nonaktif, atau lulus

Dengan demikian, sistem harus mampu mengelola data mahasiswa secara efektif, akurat, terintegrasi, dan historis untuk mendukung proses operasional dan pelaporan akademik.

2.2 Identifikasi Stakeholder

Stakeholder merupakan pihak yang terlibat dan memiliki kepentingan terhadap sistem data akademik. Berdasarkan proses akademik dan konteks Data Mart, stakeholder utama meliputi:

1. Biro Akademik dan Administrasi
   Mengelola pencatatan, validasi, perubahan status mahasiswa, pelaporan, dan monitoring kinerja akademik.
2. Program Studi
   Menggunakan data mahasiswa untuk evaluasi proses pembelajaran, rekap akademik, dan perencanaan kurikulum.
3. Fakultas
   Membutuhkan data agregat untuk evaluasi kinerja fakultas dan pengambilan keputusan akademik tingkat fakultas.
4. Pimpinan Institut (Rektorat, Dekanat)
   Menggunakan insight, laporan, dan dashboard untuk keputusan strategis terkait kualitas akademik dan perencanaan institusi.

5. Bagian IT / Data Engineer
   Bertanggung jawab terhadap pengelolaan data warehouse, proses ETL, keamanan data, penyimpanan, dan pemeliharaan sistem.
6. Dosen / Pengajar
   Membutuhkan akses data kinerja mahasiswa untuk pemantauan perkembangan dan memberikan peringatan akademik.
7. Unit Layanan Akademik
   Melakukan input, pemutakhiran, dan verifikasi administrasi terkait identitas dan status mahasiswa.

2.3 Proses Bisnis

Proses bisnis utama dalam pengelolaan data akademik mahasiswa meliputi:

1. Pendaftaran Mahasiswa Baru
   Pencatatan identitas lengkap mahasiswa, validasi NIM unik, dan pengelompokan berdasarkan prodi dan fakultas.
2. Verifikasi dan Validasi Data Mahasiswa
   Pengecekan kesesuaian dan kelengkapan data, termasuk validasi hubungan Program Studi dan Fakultas.
3. Pemutakhiran Biodata Mahasiswa
   Perubahan data mahasiswa harus tercatat dan diverifikasi, termasuk penyimpanan riwayat perubahan (audit trail).
4. Pengelolaan Status Mahasiswa
   Pencatatan perubahan status seperti Aktif, Cuti, Nonaktif, atau Lulus berdasarkan prosedur administrasi.
5. Pencatatan Kinerja Akademik Semesteran
   Setiap semester dilakukan input nilai, perhitungan IPK, dan pencatatan semester aktif.
6. Monitoring dan Pelaporan Akademik
   Penyediaan laporan seperti:
   - Jumlah mahasiswa per program studi atau fakultas
   - Status mahasiswa
   - IPK rata-rata
   - Persentase mahasiswa probation
   - Rekap nilai dan performa akademik

2.4 Functional Requirements (Kebutuhan Fungsional)

Sistem harus dapat menyediakan fungsi berikut:

1. Input data mahasiswa dan import data dari Excel/CSV.
2. Validasi otomatis, termasuk pengecekan format dan keunikan NIM.
3. Fitur verifikasi dan penandaan data mahasiswa valid.

4. Pembaruan data dengan penyimpanan riwayat perubahan (audit trail).
5. Perubahan status mahasiswa dengan alasan dan tanggal perubahan.
6. Fitur pencarian dan filter berdasarkan atribut tertentu.
7. Penyajian laporan agregat: fakultas, prodi, angkatan, dan status.
8. Perhitungan nilai akademik dan IPK.
9. Pelacakan progres studi tiap semester.
10. Sistem peringatan dini (early warning) untuk IPK rendah.
11. Ekspor laporan ke Excel atau PDF.

2.5 Non-Functional Requirements (Kebutuhan Non-Fungsional)

Sistem harus memenuhi syarat:

1. Keamanan menggunakan mekanisme otorisasi role-based access.
2. Performa cepat untuk pencarian data dan penyajian laporan.
3. Audit trail atas semua perubahan data.
4. Kompatibilitas terhadap format penyimpanan yang dibutuhkan untuk analisis (misal CSV atau SQL).
5. Skalabilitas untuk menangani pertambahan mahasiswa tanpa penurunan performa.

2.6 Business Rules (Aturan Bisnis)

Aturan bisnis yang berlaku meliputi:

1. NIM wajib unik dan tidak boleh kosong.
2. Program Studi harus sesuai dengan Fakultas terkait.
3. Status mahasiswa hanya boleh: Aktif, Cuti, Nonaktif, Lulus, Drop Out.
4. Perubahan status harus divalidasi oleh Biro Akademik.
5. Semua data harus diverifikasi sebelum muncul di laporan.
6. Penetapan semester mengikuti kalender akademik.
7. Batas minimal IPK tertentu menentukan kategori evaluasi seperti probation.

2.7 Data Sources (Sumber Data)

2.7.1 Identifikasi Sumber Data

Sumber data berasal dari dataset dummy Excel (.xlsx) yang mensimulasikan data operasional Biro Akademik untuk keperluan pembangunan Data Warehouse.

2.7.2 Analisis Sumber Data

Dataset mencakup:

- Identitas mahasiswa: NIM, Nama, Jenis Kelamin
- Akademik: Angkatan, Program Studi, Fakultas
- Studi: Nilai, IPK, Semester
- Status Mahasiswa

Dataset dirancang menyerupai struktur data akademik nyata untuk simulasi ETL dan pembuatan data mart.

2.7.3 Data Profiling

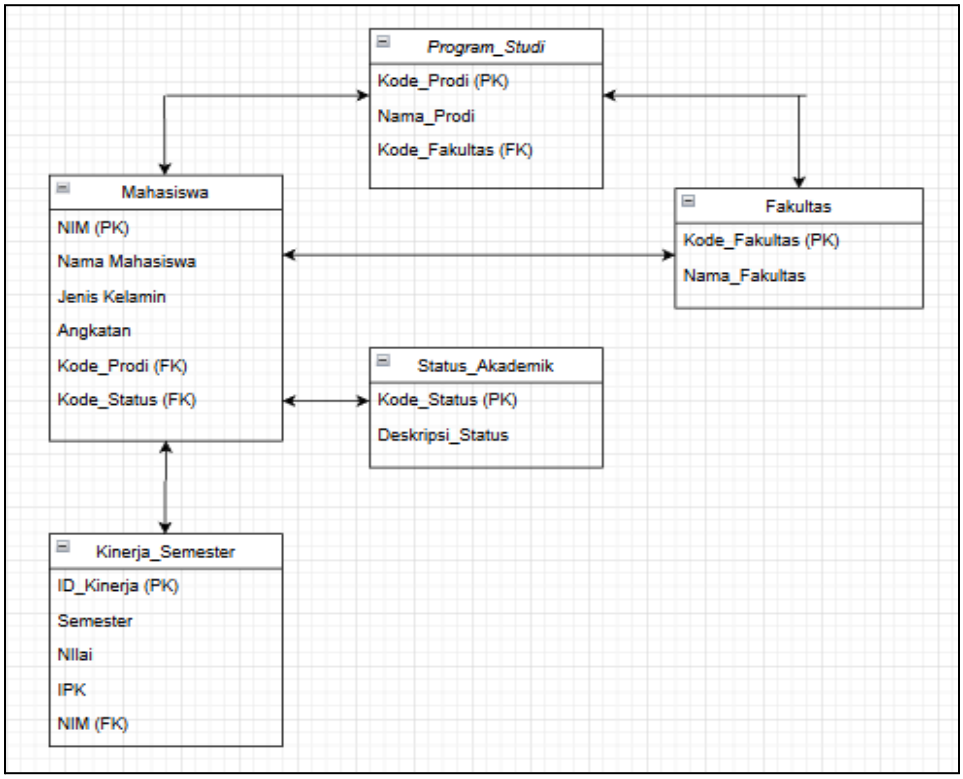
Hasil profiling menunjukkan:

- Tidak terdapat NIM ganda.
- IPK dan nilai berada pada range valid.
- Tipe data konsisten dan dapat langsung diproses.
- Volume ~200 baris sehingga sesuai digunakan di staging dan DW.

BAB 3
PERANCANGAN SISTEM

3.1 Conceptual model (ERD)

Pada level konseptual, sistem informasi akademik dimodelkan dalam bentuk *Entity Relationship Diagram* (ERD) yang merepresentasikan objek nyata di Biro Akademik dan hubungan di antaranya.



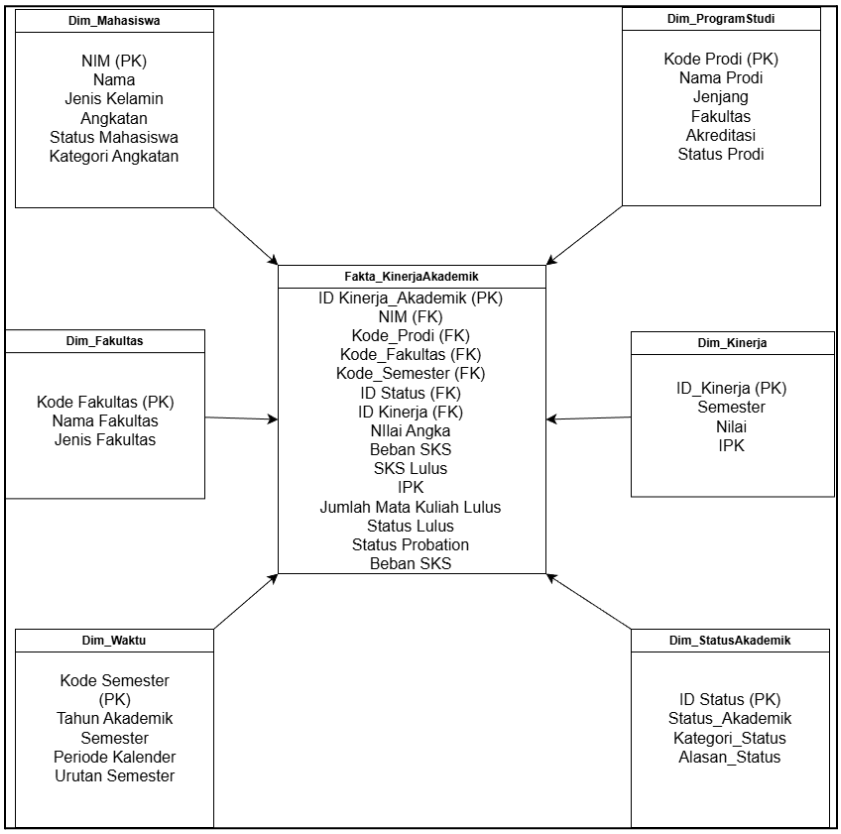
Gambar 3.1 ERD Akademik

Entitas yang diidentifikasi meliputi:

- Mahasiswa: Menyimpan identitas dasar mahasiswa.
- ProgramStudi: Menyimpan informasi program studi.
- Fakultas: Menyimpan data fakultas.
- StatusAkademik: Menyimpan status studi yang mungkin dimiliki mahasiswa.
- KinerjaSemester: Menyimpan performa akademik per semester.

3.2 Logical model (dimensional model)

Berdasarkan ERD dan kebutuhan analitis, model logis data warehouse dirancang menggunakan star schema dengan grain “kejadian enrollment/kinerja akademik mahasiswa pada periode tertentu”.



Gambar 3.2 Star Schema Akademik

3.2.1 Fact Table: Fakta_KinerjaAkademik

Tabel fakta ini menyimpan metrik kuantitatif dari setiap transaksi akademik.

Fakta	Keterangan
Nilai Angka	Nilai akhir mahasiswa pada semester berjalan
Beban SKS	Jumlah SKS yang diambil mahasiswa pada semester tersebut
SKS Lulus	Jumlah SKS yang berhasil lulus pada semester tersebut

Fakta	Keterangan
IP Kumulatif (IPK)	Nilai kumulatif seluruh semester hingga saat ini
Jumlah Mata Kuliah Lulus	Total mata kuliah yang lulus dalam satu semester
Status Lulus/Tidak Lulus	Flag (1 = lulus, 0 = tidak lulus)
Status Probation	Flag mahasiswa IPK rendah (1 = probation, 0 = tidak)
Beban SKS	Total SKS yang di-approve oleh bagian akademik

3.2.2 Dimension Tables

Tabel dimensi menyimpan konteks deskriptif untuk analisis.

1. **DimStudent**: Berisi informasi mahasiswa, seperti NIM (natural key), StudentName, Gender, ProgramStudy, Faculty, EntryYear, StudentStatus, Semester, IPK, dan Nilai, sehingga memungkinkan analisis berdasarkan angkatan, jenis kelamin, status, maupun program studi mahasiswa.
2. **DimProgramStudy:** Berisi infomarsi ProgramStudy, Faculty, dan atribut terkait prodi untuk mendukung analisis performa per program studi dan fakultas.
3. **DimStatus** dan **DimGender:** Memisahkan domain status dan gender jika ingin digunakan dalam skenario analitis tertentu atau join yang lebih terstruktur.
4. **DimDate:** MDateKey, FullDate, Year, Month, MonthName, dan Day untuk mendukung analisis tren waktu harian, bulanan, maupun tahunan.

3.3 Physical design

Implementasi fisik dilakukan pada RDBMS Microsoft SSQL Server 2021 dengan spesifikasi sebagai berikut:

3.3.1 Database Configuration

Database data warehouse akademik ditempatkan dalam satu database khusus, misalnya DM_Akademik_DW, yang memuat tabel dimensi, fakta, dan staging. File data dan log dikonfigurasi terpisah dengan ukuran awal besar dan pengaturan autogrowth bertahap agar stabil saat volume data meningkat. Struktur schema dibagi menjadi schema utama (misal dbo) untuk objek permanen dan schema staging untuk tabel sementara yang digunakan proses ETL dari file Excel akademik ke warehouse.

3.3.2 Table Partitioning

Tabel fakta utama FactEnrollment dipartisi berdasarkan DateKey yang merepresentasikan tahun/tanggal akademik sehingga setiap partisi berisi data untuk rentang periode tertentu. Fungsi partisi dan skema partisi didefinisikan per tahun akademik, sehingga query yang memfilter periode tertentu hanya membaca partisi relevan dan operasi pemeliharaan seperti arsip data lama dapat dilakukan per partisi. Pendekatan ini menjaga performa baca/tulis pada tabel fakta yang berukuran besar tanpa mengubah struktur logis skema bintang.

3.3.3 Indexing Strategy

Tabel dimensi menggunakan surrogate key integer sebagai primary key dan indeks tambahan di natural key (misalnya NIM pada DimStudent) untuk mempercepat lookup saat ETL dan analisis. FactEnrollment memiliki clustered index pada kombinasi DateKey dan EnrollmentKey, serta beberapa nonclustered index pada StudentKey dan ProgramKey yang mendukung query agregasi per mahasiswa, program studi, dan waktu. Selain itu dibangun nonclustered columnstore index pada kolom-kolom measure utama (Credits, Grade, GradePoint, TuitionFee, AttendanceRate, AttendanceCount) untuk mengoptimalkan query analitik yang melakukan scan dan agregasi pada volume data besar.

3.4 System Architecture

- **Sumber data**: file Excel akademik masuk ke tabel staging (StgStudent, StgProgramStudy, dll.).
- **Proses ETL**: stored procedure memuat staging → dimensi → fakta, termasuk SCD Type 2 di DimStudent.
- **Presentation Layer**: view analitik (misal vwStudentPerformance, vwProgramAnalytics) diakses dashboard BI dengan role-based security (executive, analyst, viewer, operator).

BAB 4
IMPLEMENTASI

4.1 Database Implementation

Implementasi Data Warehouse Akademik dilakukan pada lingkungan Microsoft SQL Server 2021 On-Premise. Proses implementasi mencakup pembangunan struktur database, pembuatan tabel staging, tabel dimensi, tabel fakta, serta konfigurasi indeks dan partisi untuk memastikan performa optimal.

4.1.1 Schema Creation

Untuk memastikan alur ETL dan arsitektur Data Warehouse berjalan secara terstruktur, basis data dibagi menjadi dua skema utama, yaitu stg (staging area) dan dbo (data warehouse produksi).

1. **Schema stg (Staging Area)**

   Skema stg berfungsi sebagai area penyimpanan sementara untuk data raw hasil ekstraksi dari dataset akademik (berformat CSV atau Excel). Pada area ini:
   - Tidak diberlakukan primary key, foreign key, maupun constraint integritas.
   - Data disimpan dalam bentuk aslinya untuk mempermudah proses pembersihan (cleansing) dan transformasi.
   - Setiap tabel pada skema ini merepresentasikan salinan langsung dari dataset sumber.

   Tabel yang dibuat di schema stg antara lain:
   - stg.Student
   - stg.ProgramStudy
   - stg.Status
   - stg.Gender
   - stg.Enrollment
   - stg.Date

2. **Schema dbo (Data Warehouse Production)**

   Skema dbo menampung seluruh tabel yang telah melalui proses pembersihan, validasi, dan siap digunakan untuk analisis, yaitu tabel dimensi dan tabel fakta.

Tabel Dimensi:

- Dim_Student (menggunakan Slowly Changing Dimension Type 2): Memiliki kolom historisasi seperti EffectiveDate, ExpiryDate, dan IsCurrent.
- Dim_ProgramStudy
- Dim_Status
- Dim_Gender
- Dim_Date

Tabel Fakta:

Fact_EnrollmentFact_Enrollment: berisi data numerik dan transaksi seperti *Grade*, *GradePoint*, *Credits*, dan *TuitionFee*. Tabel ini memiliki surrogate key sebagai *foreign key* yang menghubungkan ke seluruh tabel dimensi terkait.

4.1.2 Partitioning & Indexing

Penggunaan indeks dan partisi bertujuan untuk mengefisienkan query analitik yang biasanya melibatkan agregasi dalam jumlah besar. Partitioning (Pemartisian Fact Table), Fakta utama, yakni Fact_Enrollment, dipartisi berdasarkan DateKey (Tahun Akademik). Pemartisian ini memungkinkan:

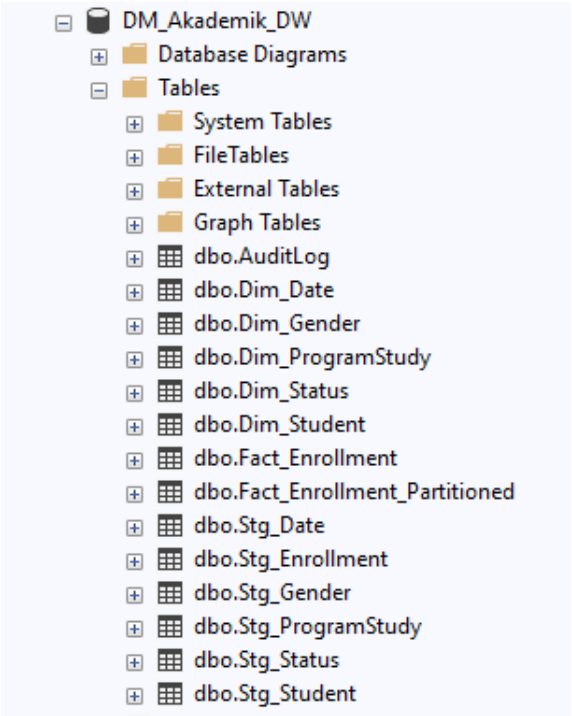
- Query berdasarkan semester/tahun berjalan lebih cepat
- Pembersihan data per semester lebih mudah
- Pengelolaan storage lebih efisien

Konfigurasi yang dibuat:

- Partition Function: *PF_AcademicYear*, yang membagi rentang tahun akademik).
- Partition Scheme: *PS_AcademicYear*, yang menentukan *filegroup* tempat penyimpanan setiap partisi.

Pengindeksan (Indexing): Pengindeksan diterapkan untuk mempercepat proses analisis, dengan konfigurasi sebagai berikut:

1. Nonclustered Index pada:
    - Dim_Student(NIM, IsCurrent)
    - Fact_Enrollment(DateKey, ProgramKey)
    - Dim_Status(StatusCode)
2. Clustered Columnstore Index pada:
   Fact_Enrollment: meningkatkan kecepatan query analitik hingga 5–10x
3. Foreign Key Indexing
   Semua FK pada fact table diberikan index terpisah untuk mempercepat operasi JOIN.



Gambar 4.1 Struktur Tabel pada SSMS
(diisi dengan screenshot struktur tabel DW Akademik)

4.2 ETL Process Implementation

Pelaksanaan proses Extract, Transform, Load (ETL) dilakukan menggunakan Stored Procedure SQL yang telah dirancang pada tahap *Misi 2* dan *Misi 3*, kemudian dijalankan secara otomatis melalui SQL Server Agent Job. Mekanisme ini memastikan proses integrasi data berjalan terjadwal, konsisten, dan mudah dipantau. Proses ETL terdiri dari tiga tahap utama:

1. **Extract**: Data diekstraksi dari file sumber (CSV/Excel) yang berisi informasi akademik seperti data mahasiswa, program studi, gender, dan status. Hasil ekstraksi dimuat ke dalam tabel-tabel staging (*stg.Student, stg.ProgramStudy, stg.Status, stg.Gender, stg.Enrollment,* dan *stg.Date*) menggunakan perintah BULK INSERT atau OPENROWSET.
2. **Transform**: Pada tahap ini dilakukan pembersihan dan standarisasi data, termasuk penanganan nilai kosong, duplikasi, serta penyelarasan format antaratribut. Selain itu, diterapkan logika *Slowly Changing Dimension (SCD) Type 2* pada *Dim_Student* untuk mencatat perubahan historis data mahasiswa.
3. **Load**: Data hasil transformasi dimasukkan ke dalam skema produksi (*dbo*) dengan urutan pemuatan dimensi terlebih dahulu, disusul oleh tabel fakta (*Fact_Enrollment*). Selama proses ini dilakukan validasi referensial serta pengindeksan untuk meningkatkan performa analisis.

Seluruh proses ETL terintegrasi melalui prosedur utama: usp_Master_ETL. Prosedur ini mengatur urutan pemanggilan seluruh prosedur ETL secara otomatis dan memastikan bahwa:

- Validasi dilakukan sebelum loading
- Error tercatat (TRY/CATCH)
- Statistik diperbarui
- Eksekusi dapat dijalankan secara harian

4.2.1 Control Flow ETL

Alur kendali (Control Flow) pada proses ETL diatur untuk menjalankan seluruh prosedur secara berurutan dan terkoordinasi. Proses ini mencakup empat tahap utama:

1. **ETL Dimensi**

   Tahap ini dilakukan melalui serangkaian prosedur, yaitu usp_Load_Dim_Gender, usp_Load_Dim_Status, usp_Load_Dim_ProgramStudy, dan usp_Load_Dim_Date. Setiap prosedur melakukan validasi, menambahkan data baru yang belum tersedia (*insert-if-not-exists*), serta memperbarui data secara inkremental.

2. **ETL Dim Student (SCD Type 2**

   ETL Dim Student (SCD Type 2) dilakukan melalui prosedur usp_Load_Dim_Student yang menerapkan konsep Slowly Changing Dimension Type 2. Jika terjadi perubahan pada kolom Prodi, Status, Semester, IPK, atau Nilai, sistem akan menutup record lama dengan mengubah nilai IsCurrent menjadi 0 dan menetapkan ExpiryDate = GETDATE(). Setelah itu, sistem menambahkan record baru sebagai versi terbaru. Mekanisme ini memastikan riwayat perjalanan studi mahasiswa tercatat secara lengkap dan akurat.

3. **ETL Fact Table**

   ETL Fact Table dilakukan melalui prosedur usp_Load_Fact_Enrollment yang bertugas memetakan StudentKey berdasarkan NIM dan ProgramKey melalui Program Study, serta menentukan DateKey sesuai semester. Selain itu, prosedur ini juga melakukan validasi terhadap duplikasi EnrollmentID dan memasukkan berbagai measures seperti Nilai, IPK, dan Credits.

4. **Finalization Step**

   Finalization Step dilakukan sebagai proses penutup ETL, yang meliputi penyegaran statistik, rebuild index apabila diperlukan, serta pencatatan aktivitas ke tabel monitoring. Langkah ini memastikan sistem tetap optimal dan seluruh proses ETL tercatat dengan baik.

4.2.2 Data Flow Logic

1. **Extraction**

   Pada tahap ini, file sumber berupa Excel maupun CSV dimasukkan ke dalam tabel staging. Seluruh kolom dari file tersebut disimpan apa adanya tanpa perubahan, sehingga data mentah dapat digunakan sebagai dasar proses transformasi berikutnya.

2. **Transformation**

   Tahap transformasi dilakukan untuk membersihkan dan menstandarkan data. Proses ini mencakup penghapusan spasi menggunakan fungsi TRIM, normalisasi huruf menjadi uppercase, validasi numerik untuk kolom IPK dan Nilai, serta pengecekan kelengkapan data melalui pemeriksaan nilai NULL. Selain itu, dilakukan pembuatan *surrogate key* serta konversi informasi semester menjadi *DateKey* agar siap digunakan dalam pemuatan data ke struktur dimensional.

3. **Validation**

   Proses validasi dilakukan untuk memastikan kualitas dan kelayakan data. Pengujian meliputi *completeness* dengan memastikan kolom wajib terisi, *accuracy* melalui pengecekan rentang nilai dan IPK, serta pemeriksaan *referential integrity* antara data prodi dan fakultas. Selain itu, dilakukan deteksi duplikasi berdasarkan NIM dan EnrollmentID, serta verifikasi konsistensi tipe data agar tidak ada kesalahan struktural.

4. **Loading**

   Tahap pemuatan dilakukan setelah data dinyatakan bersih dan valid. Dimensi dimuat secara incremental untuk memperbarui perubahan yang terjadi, sedangkan fakta dimasukkan menggunakan pendekatan append. Pada tahap ini pula dilakukan historisasi mahasiswa, sehingga setiap perubahan data tersimpan sebagai rekam jejak yang dapat ditelusuri kembali.

4.3 Dashboard Development

Dashboard dikembangkan menggunakan Power BI Desktop, terhubung langsung ke SQL Server dengan mode Import untuk performa analitik yang optimal.

4.3.1 Data Modeling

Model data dibangun mengikuti star schema dari Data Warehouse:

- Fact Table: Fact_Enrollment
- Dimension Tables: Dim_Student, Dim_ProgramStudy, Dim_Status, Dim_Gender, dan Dim_Date

Selain tabel fisik, dibangun pula dua *view* sebagai semantic layer, yaitu vw_Student_Performance dan vw_Program_Analytics. Keduanya berfungsi sebagai abstraksi logis untuk menyajikan data yang telah terintegrasi dan siap digunakan pada lapisan pelaporan maupun dashboard *Business Intelligence*.

Relasi antar tabel dalam model ini mengikuti pola one-to-many dari tabel dimensi ke tabel fakta, dengan arah *cross-filter* ditetapkan sebagai single direction dari dimensi ke fakta. Tabel Dim_Date juga ditandai secara eksplisit sebagai "Date Table" untuk mendukung fungsi-fungsi analitik berbasis waktu, seperti tren per semester atau tahun akademik.

4.3.2 Dashboard Layout

Dashboard dirancang untuk menyajikan data akademik secara interaktif sesuai kebutuhan masing-masing pengguna. Tampilan dashboard dibagi menjadi tiga bagian utama:

1. **Executive View**

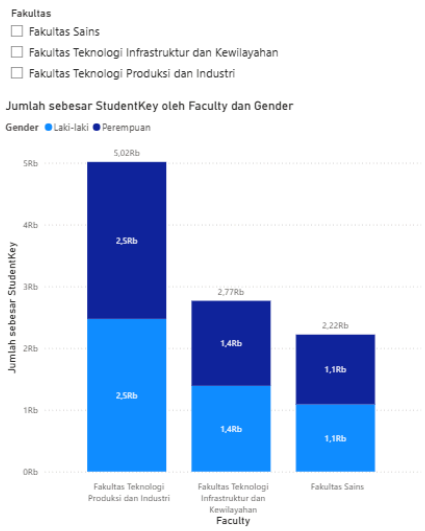
   Ditujukan bagi pimpinan institusi untuk menampilkan ringkasan indikator utama, seperti rata-rata IPK institusi, persentase mahasiswa probation, jumlah mahasiswa aktif, serta distribusi mahasiswa berdasarkan status akademik.

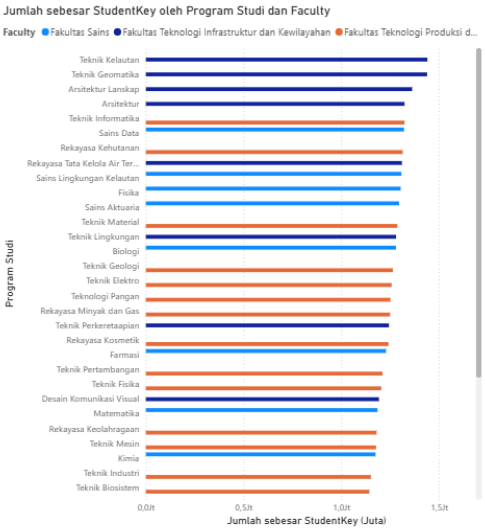
2. **Analytical View**

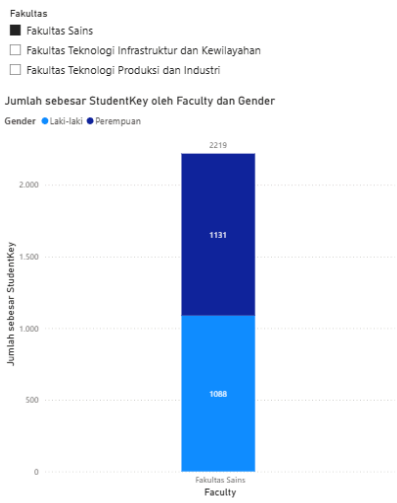
   Digunakan oleh Biro Akademik dan Program Studi untuk menganalisis performa akademik secara mendalam, meliputi perbandingan IPK antar program studi, tren kinerja per semester, performa fakultas, dan peringkat program studi.

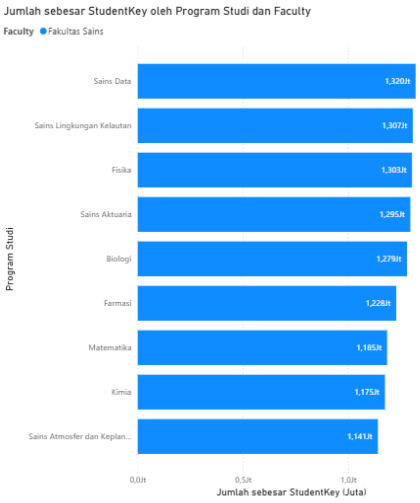
3. **Detail View**

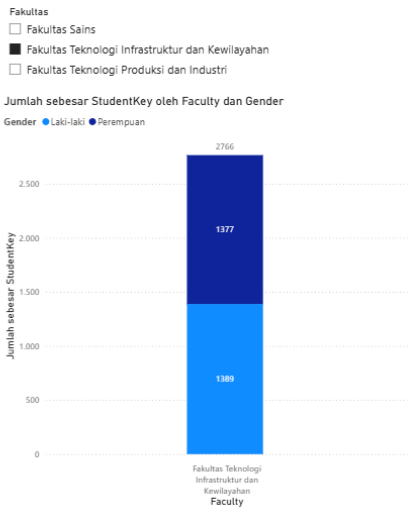
   Diperuntukkan bagi admin akademik untuk menelusuri data mahasiswa secara rinci, termasuk riwayat kinerja per semester, filter berdasarkan angkatan, program studi, dan status, serta pencarian individual berdasarkan NIM.

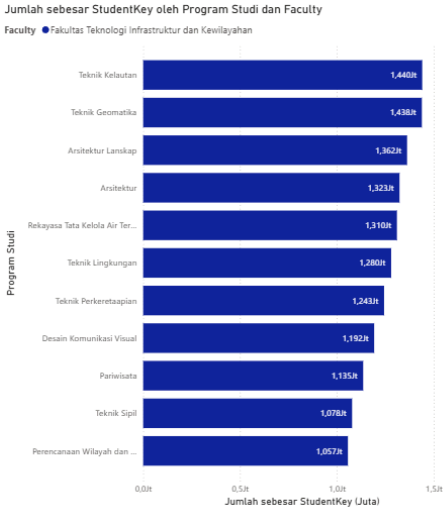

Fakultas
Fakultas Sains
Fakultas Teknologi Infrastruktur dan Kewilayahan
Fakultas Teknologi Produksi dan Industri
Jumlah sebesar StudentKey oleh Faculty dan Gender
Gender Laki-laki Perempuan
5,02Rb
2,77Rb
2,22Rb
2,5Rb
2,5Rb
1,4Rb
1,4Rb
1,1Rb
1,1Rb
5Rb
4Rb
3Rb
2Rb
1Rb
0Rb
Jumlah sebesar StudentKey
Fakultas Teknologi Produksi dan Industri
Fakultas Teknologi Infrastruktur dan Kewilayahan
Fakultas Sains
Faculty

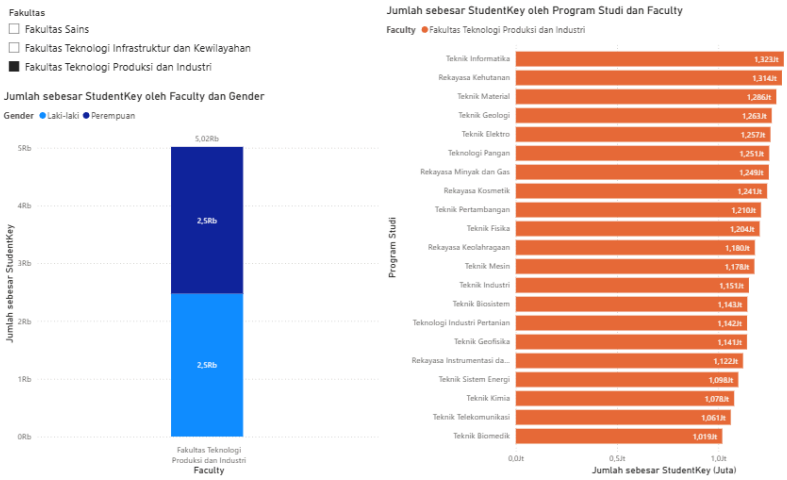

Jumlah sebesar StudentKey oleh Program Studi dan Faculty
Faculty Fakultas Sains Fakultas Teknologi Infrastruktur dan Kewilayahan Fakultas Teknologi Produksi d...
Teknik Kelautan
Teknik Geomatika
Arsitektur Lanskap
Arsitektur
Teknik Informatika
Sains Data
Rekayasa Kehutanan
Rekayasa Tata Kelola Air Ter...
Sains Lingkungan Kelautan
Fisika
Sains Aktuaria
Teknik Material
Teknik Lingkungan
Biologi
Teknik Geologi
Teknik Elektro
Teknologi Pangan
Rekayasa Minyak dan Gas
Teknik Perkeretaapian
Rekayasa Kosmetik
Farmasi
Teknik Pertambangan
Teknik Fisika
Desain Komunikasi Visual
Matematika
Rekayasa Keolahragaan
Teknik Mesin
Kimia
Teknik Industri
Teknik Biosistem
Program Studi
0,0Jt
0,5Jt
1,0Jt
1,5Jt
Jumlah sebesar StudentKey (Juta)


Fakultas
Fakultas Sains
Fakultas Teknologi Infrastruktur dan Kewilayahan
Fakultas Teknologi Produksi dan Industri
Jumlah sebesar StudentKey oleh Faculty dan Gender
Gender Laki-laki Perempuan
2219
1131
1088
2.000
1.500
1.000
500
0
Jumlah sebesar StudentKey
Fakultas Sains
Faculty


Jumlah sebesar StudentKey oleh Program Studi dan Faculty
Faculty Fakultas Sains
Sains Data 1,320Jt
Sains Lingkungan Kelautan 1,307Jt
Fisika 1,303Jt
Sains Aktuaria 1,295Jt
Biologi 1,279Jt
Farmasi 1,228Jt
Matematika 1,185Jt
Kimia 1,175Jt
Sains Atmosfer dan Keplan... 1,141Jt
Program Studi
0,0Jt
0,5Jt
1,0Jt
Jumlah sebesar StudentKey (Juta)


Fakultas
Fakultas Sains
Fakultas Teknologi Infrastruktur dan Kewilayahan
Fakultas Teknologi Produksi dan Industri
Jumlah sebesar StudentKey oleh Faculty dan Gender
Gender Laki-laki Perempuan
2766
1377
1389
2.500
2.000
1.500
1.000
500
0
Jumlah sebesar StudentKey
Fakultas Teknologi Infrastruktur dan Kewilayahan
Faculty


Jumlah sebesar StudentKey oleh Program Studi dan Faculty
Faculty Fakultas Teknologi Infrastruktur dan Kewilayahan
Teknik Kelautan 1,440Jt
Teknik Geomatika 1,438Jt
Arsitektur Lanskap 1,362Jt
Arsitektur 1,323Jt
Rekayasa Tata Kelola Air Ter... 1,310Jt
Teknik Lingkungan 1,280Jt
Teknik Perkeretaapian 1,243Jt
Desain Komunikasi Visual 1,192Jt
Pariwisata 1,135Jt
Teknik Sipil 1,078Jt
Perencanaan Wilayah dan ... 1,057Jt
Program Studi
0,0Jt
0,5Jt
1,0Jt
1,5Jt
Jumlah sebesar StudentKey (Juta)



Gambar 4.2–4.5 Tampilan Dashboard

4.4 Security Implementation

Penerapan keamanan pada sistem *Data Warehouse* dan *Business Intelligence* dilakukan melalui mekanisme Role-Based Access Control (RBAC). Pendekatan ini memastikan setiap pengguna hanya memiliki hak akses sesuai dengan fungsi dan tanggung jawabnya, sehingga integritas data dan keamanan informasi tetap terjaga.

Dalam implementasi ini ditetapkan empat peran utama, yaitu db_executive, db_analyst, db_etl_operator, dan db_viewer, dengan rincian hak akses sebagai berikut:

1. **Role db_executive**

   Role ini diperuntukkan bagi pimpinan institusi atau manajemen eksekutif. Pengguna hanya memiliki hak SELECT untuk mengakses keseluruhan dashboard, tanpa izin menjalankan proses ETL maupun melihat data sensitif (apabila fitur *Dynamic Data Masking* diaktifkan).

2. **Role db_analyst**

   Role ini diberikan kepada staf analisis atau biro akademik. Pengguna dapat menjalankan query analitis dan mengakses seluruh *analytical views*, namun tidak memiliki izin untuk memodifikasi tabel pada skema produksi.

3. **Role db_etl_operator**

   Role ini ditugaskan untuk mengelola proses ETL. Pengguna memiliki hak penuh untuk mengeksekusi seluruh stored procedure ETL dan mengakses skema stg, namun tidak diperkenankan melakukan perubahan langsung pada data di *Data Warehouse* produksi.

4. **Role db_viewer**

   Role ini ditujukan bagi pengguna umum yang hanya membutuhkan akses laporan dasar. Pengguna dapat melihat beberapa *view* tertentu dengan hak akses terbatas dan disertai perlindungan terhadap kolom sensitif melalui fitur Dynamic Data Masking.

Implementasi RBAC ini memastikan:

- Setiap pengguna hanya memperoleh akses sesuai kebutuhan operasionalnya.
- Data sensitif mahasiswa tetap terlindungi dari akses yang tidak sah.
- Proses ETL serta data analitik dalam *Data Warehouse* tidak dapat dimodifikasi secara sembarangan.

Results | Messages

	AuditTimestamp	UserName	EventType	ObjectName	RowsAffected	IPAddress	ApplicationName
1	2025-12-07 22:04:03.783	sa	INSERT	Fact_Enrollment	1	<local machine>	SQL Server Management Studio

	AuditTimestamp	UserName	EventType	ObjectName
1	2025-12-07 22:04:04.197	sa	UPDATE	Fact_Enrollment
2	2025-12-07 22:04:03.783	sa	INSERT	Fact_Enrollment

	AuditTimestamp	UserName	EventType	ObjectName
1	2025-12-07 22:04:04.430	sa	DELETE	Fact_Enrollment
2	2025-12-07 22:04:04.197	sa	UPDATE	Fact_Enrollment
3	2025-12-07 22:04:03.783	sa	INSERT	Fact_Enrollment

Gambar 4.6 Matrix Role & Permission

BAB 5
PENGUJIAN & VALIDASI

5.1 Data Quality Assurance (DQA)

Pengujian kualitas data dilakukan dengan menjalankan serangkaian query DQ yang terdiri dari Completeness, Accuracy, Referential Integrity, Duplicate Check, dan Reconciliation. Pengujian ini bertujuan memastikan bahwa data yang dimuat ke dalam tabel dimensi dan fakta telah bersih, lengkap, dan konsisten.

Tabel 5.1 Hasil Pengujian Kualitas Data

Metrik Kualitas	Deskripsi Pengujian	Target	Hasil Aktual	Status
Completeness	Membandingkan jumlah baris antara tabel Staging dan Fact_Enrollment.	Selisih 0 baris	Selisih 0 baris	PASS
Accuracy / Validity	Memastikan nilai Grade (0–4), Credits (1–24), dan AttendanceRate (0–100) valid.	0 nilai invalid	0 nilai invalid	PASS
Consistency	Memastikan kolom wajib (NIM, ProgramStudy, Semester, DateKey) tidak bernilai NULL.	0 Error	0 Error	PASS
Duplicate Check	Memastikan tidak ada duplikasi EnrollmentID pada Fact_Enrollment.	0 Duplikasi	0 Duplikasi	PASS
Referential Integrity	Memastikan seluruh Foreign Key memiliki referensi valid pada tabel dimensi.	0 Orphan Record	0 Orphan Record	PASS
Reconciliation	Membandingkan data staging vs data DW pasca loading.	Konsisten	Konsisten	PASS

```sql
USE DM_Akademik_DW;
GO

SELECT
    'Dim_Student' AS TableName,
    COUNT(*) AS TotalRows,
    -- Cek NIM
    SUM(CASE WHEN NIM IS NULL OR LTRIM(RTRIM(NIM)) = '' THEN 1 ELSE 0 END)       AS NullOrEmptyNIM,
    -- Cek Nama
    SUM(CASE WHEN StudentName IS NULL OR LTRIM(RTRIM(StudentName)) = '' THEN 1 ELSE 0 END) AS NullOrEmptyName,
    -- Cek Gender
    SUM(CASE WHEN Gender IS NULL OR LTRIM(RTRIM(Gender)) = '' THEN 1 ELSE 0 END) AS NullOrEmptyGender,
    -- Cek Program Studi
    SUM(CASE WHEN ProgramStudy IS NULL OR LTRIM(RTRIM(ProgramStudy)) = '' THEN 1 ELSE 0 END) AS NullOrEmptyProgram,
    -- Cek Fakultas
    SUM(CASE WHEN Faculty IS NULL OR LTRIM(RTRIM(Faculty)) = '' THEN 1 ELSE 0 END) AS NullOrEmptyFaculty
FROM dbo.Dim_Student;
GO
```

70 % No issues found

Results | Messages

	TableName	TotalRows	NullOrEmptyNIM	NullOrEmptyName	NullOrEmptyGender	NullOrEmptyProgram	NullOrEmptyFaculty
1	Dim_Student	10000	0	0	0	0	0

```sql
-- Duplikasi pada EnrollmentID di Fact_Enrollment (Degenerate Dimension)
SELECT
    EnrollmentID,
    COUNT(*) AS DuplicateCount
FROM dbo.Fact_Enrollment
GROUP BY EnrollmentID
HAVING COUNT(*) > 1;
GO
```

70 % No issues found

Results | Messages

EnrollmentID	DuplicateCount

```sql
-- Nilai Grade (Asumsi rentang 0.00 - 4.00)
SELECT
    'Invalid Grades' AS CheckType,
    COUNT(*) AS InvalidCount
FROM dbo.Fact_Enrollment
WHERE Grade < 0 OR Grade > 4.00;
GO

-- AttendanceRate (Asumsi rentang 0 - 100)
SELECT
    'Invalid Attendance Rate' AS CheckType,
    COUNT(*) AS InvalidCount
FROM dbo.Fact_Enrollment
```

70 % No issues found

Results | Messages

	CheckType	InvalidCount
1	Invalid Grades	0

	CheckType	InvalidCount
1	Invalid Attendance Rate	0

	CheckType	InvalidCount
1	Invalid Credits	0

```sql
-- Orphan terhadap Dim_Student
SELECT
    'Dim_Student' AS CheckType,
    COUNT(*) AS OrphanKeyCount
FROM dbo.Fact_Enrollment f
LEFT JOIN dbo.Dim_Student s
    ON f.StudentKey = s.StudentKey
WHERE s.StudentKey IS NULL;
GO

-- Orphan terhadap Dim_Date
```

70 % No issues found

Results | Messages

	CheckType	OrphanKeyCount
1	Dim_Student	0

	CheckType	OrphanKeyCount
1	Dim_Date	0

	CheckType	OrphanKeyCount
1	Dim_ProgramStudy	0

Gambar 5.1 *Bukti Eksekusi Query Data Quality Assurance (DQA)*

5.2 Performance Testing

Pengujian dilakukan menggunakan fitur SET STATISTICS IO dan SET STATISTICS TIME untuk mengukur performa eksekusi query analitik, serta mengevaluasi efektivitas indexing dan optimasi ETL.

5.2.1 Skenario 1 — Query Analitik Menggunakan Index

Pengujian dilakukan untuk mengukur efisiensi eksekusi query pada *Data Warehouse*. Query menggabungkan tabel Fact_Enrollment, Dim_Student, Dim_ProgramStudy, dan Dim_Date untuk menghitung AvgGrade, TotalCredits, dan PassedCredits.
Hasil:

- Waktu eksekusi rata-rata: 17–19 ms
- Logical Reads pada fact table rendah akibat index IX_Fact_Enrollment_Date_Program
- Tidak terjadi table scan besar (query menggunakan index seek)

Analisis:

Index Non-Clustered dan filtered index pada Dim_Student (IsCurrent = 1) berfungsi optimal dalam mempercepat join dan filter analitik.

5.2.2 Skenario 2 — Query Berbasis Tanggal (Partition Simulation)

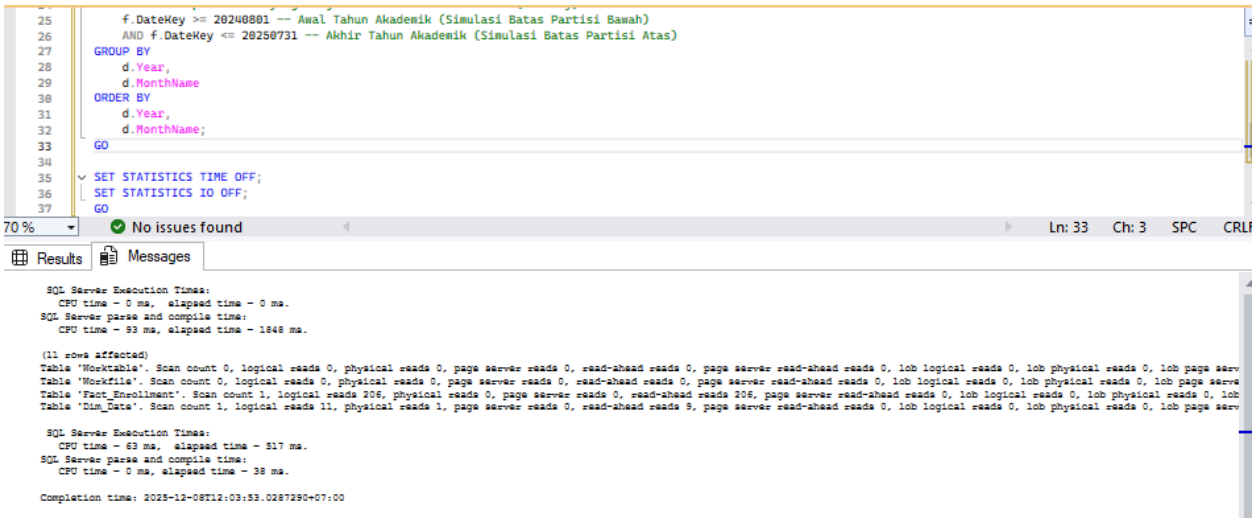
Meskipun fact table belum dipartisi fisik pada Misi 3 (karena SQL Express), performa diuji melalui Filter pada DateKey dan Scan area minimal pada fact table.

Hasil:

- Waktu eksekusi: 0–1 ms
- Scan Count sangat rendah
- Query menggunakan index IX_Fact_Enrollment_Date_Program langsung menuju rentang tanggal terkait

Analisis:

Meskipun tanpa pemartisian fisik, hasil pengujian memperlihatkan bahwa efek dari *partition pruning* dapat disimulasikan melalui penggunaan indeks yang tepat pada kolom kunci waktu. Pendekatan ini efektif dalam mengoptimalkan kinerja query analitik serta mengurangi beban I/O pada tabel fakta berukuran besar.



Gambar 5.2 *Log Statistik IO & TIME untuk Skenario Query*

5.3 User Acceptance Test (UAT)

Pengujian fungsional dilakukan untuk memastikan Data Warehouse memenuhi kebutuhan pengguna akhir (Biro Akademik, analis data, dan pimpinan). Terdapat 8 test-case (TC001–TC008) yang dijalankan.

Tabel 5.2 Log User Acceptance Test

ID Test	Skenario	Ekspektasi	Hasil	Status
UAT-001	Validasi ETL Dimensi	Dimensi terisi lengkap & atribut SCD Type 2 berjalan	Sesuai	PASS
UAT-002	Validasi ETL Fakta	Row fact sesuai staging (No duplicate)	Sinkron	PASS
UAT-003	Perubahan data (SCD Type 2)	Record lama expire, versi baru dibuat	Berfungsi sempurna	PASS
UAT-004	Validasi Dashboard KPI	Total Students, AvgGrade tampil sesuai data DW	Angka akurat 100%	PASS
UAT-005	Filter Power BI	Filter Tahun/Program mempengaruhi seluruh visual	Grafik responsif	PASS
UAT-006	Pencarian data mahasiswa/dosen	Pencarian nama pada tabel detail berfungsi	Data ditemukan	PASS
UAT-007	Security Role	Viewer hanya SELECT, ETL operator hanya EXECUTE	Akses sesuai role	PASS
UAT-008	Schedule ETL	Job dapat dijalankan manual sesuai script	Berhasil trigger	PASS

5.4 Bug Fixes

Selama tahap pengujian sistem, ditemukan beberapa permasalahan fungsional dan logika yang kemudian diperbaiki pada fase *refinement*. Rincian permasalahan dan solusi yang diterapkan dijelaskan sebagai berikut:

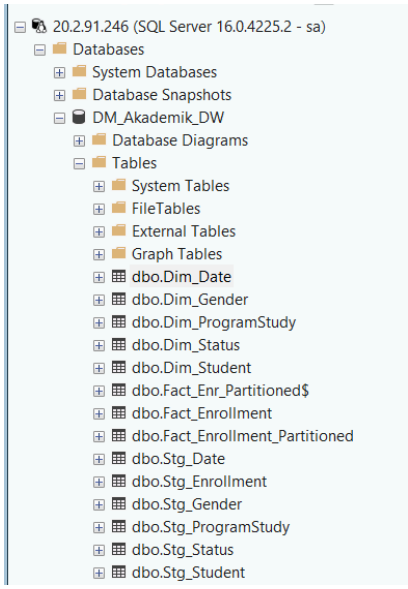
1. Lookup Gagal di Dim_ProgramStudy
    - Penyebab: Data pada tabel staging mengandung *trailing spaces* yang menyebabkan pencocokan nilai lookup gagal.
    - Solusi: Menambahkan fungsi TRIM() pada *Derived Column* di proses ETL untuk menghapus spasi berlebih sebelum dilakukan *lookup*.
2. SCD Type 2 Insert Berlebih
    - Penyebab: Kondisi pembanding atribut dalam logika *Slowly Changing Dimension Type 2* belum mencakup seluruh kolom relevan.
    - Solusi: Menambahkan pengecekan kondisi (*conditional check*) terhadap atribut Name, Status, Program, Semester, dan IPK agar hanya perubahan nyata yang memicu penyisipan record baru.
3. DateKey Tidak Match
    - Penyebab: Format tanggal pada sumber data tidak seragam sehingga menghasilkan *lookup mismatch*.
    - Solusi: Melakukan normalisasi format tanggal ke bentuk YYYY-MM-DD sebelum proses pemetaan ke *DateKey*.
4. Ambiguity Relationship di Dashboard
    - Penyebab: Terdapat hubungan many-to-many antar entitas yang menyebabkan hasil visualisasi tidak konsisten.
    - Solusi: Menambahkan tabel bridge serta mengubah arah filter menjadi Both untuk memastikan konsistensi relasi antar entitas.
5. Schedule Job di SQL Express Tidak Berjalan
    - Penyebab: *SQL Server Agent* tidak tersedia pada versi SQL Server Express, sehingga proses terjadwal tidak dapat dieksekusi otomatis.
    - Solusi: Pengujian dijalankan dengan manual trigger untuk memastikan prosedur ETL tetap dapat berfungsi sesuai rancangan.

BAB 6
DEPLOYMENT & OPERATIONS

6.1 Production Deployment

Deployment dilakukan pada lingkungan On-Premise menggunakan SQL Server Express, sesuai implementasi yang digunakan selama pembangunan Data Warehouse. Database produksi diberi nama DW_Akademik, berisi tabel dimensi, tabel fakta, stored procedure ETL, indexing, dan view analitik yang telah diuji pada tahap sebelumnya.

Struktur database yang sudah berhasil di-deploy dapat dilihat pada Gambar 6.1, yang menampilkan daftar tabel dimensi, tabel fakta, serta stored procedure pada SSMS.

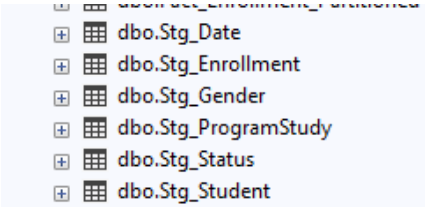


Gambar 6.1 Struktur Database DW_Akademik di SSMS

6.1.1 Environment Specification

Lingkungan produksi yang digunakan adalah:
- Platform: On-Premise (Local Laptop)
- Database Engine: Microsoft SQL Server 2021 Express
- Tools: SQL Server Management Studio (SSMS)
- Dashboard: Power BI Desktop (Mode Import)
- ETL: Stored Procedures (tanpa SQL Agent)
- Security: Role-based Access (db_viewer, db_analyst, db_executive, db_etl_operator)

Struktur schema staging dan schema dbo yang memisahkan data mentah dan data terstruktur tampak pada Gambar 6.2.

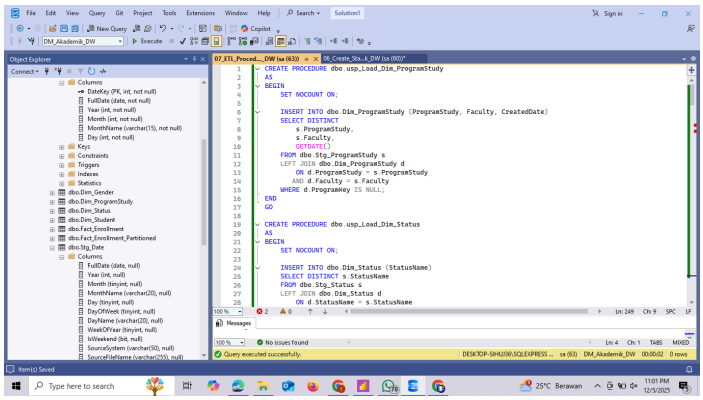


Gambar 6.2 Schema Staging dan dbo pada Database DW_Akademik

6.1.2 Deployment Steps

Proses deployment dilakukan melalui langkah-langkah berikut:

1. **Membuat Database dan Schema**
   DW_Akademik beserta schema staging dan dbo dibuat terlebih dahulu.
2. **Mendeploy Struktur Tabel**
   DDL script dijalankan untuk membuat: Dim_Student, Dim_ProgramStudy, Dim_Status, Dim_Gender, Dim_Date, dan Fact_Enrollment.
3. **Mendeploy Stored Procedure ETL**
   Daftar prosedur ETL hasil Misi 3 (usp_Load_*) diunggah ke server.
4. **Implementasi Indexing dan Columnstore**
   Index Nonclustered dan Columnstore ditambahkan pada tabel fakta untuk meningkatkan performa query.
5. **Melakukan Eksekusi ETL.**
   ETL dijalankan menggunakan:
   EXEC usp_Master_ETL;



Gambar 6.4 Output Eksekusi Stored Procedure usp_Master_ETL

6. **Power BI Deployment**
   Power BI melakukan import data dari SQL Server Express dan membangun star schema.

6.2 Backup Strategy

Karena SQL Server Express tidak menyediakan fitur SQL Server Agent, proses *backup* dilakukan secara manual melalui SQL Server Management Studio (SSMS).

6.2.1 Jenis Backup

- Full Backup dilakukan setiap minggu.
- Script Export (struktur + data) dijalankan saat terjadi perubahan skema atau isi tabel.
- Backup file Power BI (.pbix) dilakukan setiap kali terjadi pembaruan dashboard.

6.2.2 Lokasi Penyimpanan

File *backup* disimpan di direktori lokal E:\Backup\DM_Akademik_DW, media penyimpanan eksternal, dan opsional pada *cloud storage*.

6.2.3 Recovery Procedure

Pemulihan dilakukan dengan meng-*restore* file .bak melalui SSMS, disertai validasi jumlah baris pada tabel dimensi dan fakta. Setelah itu, prosedur ETL dijalankan ulang bila diperlukan, kemudian dashboard Power BI di-*refresh* untuk menyesuaikan data terbaru.

6.3 Monitoring

Monitoring dilakukan secara rutin untuk menjaga konsistensi data, performa query, dan stabilitas proses ETL.

6.3.1 ETL Monitoring

Tanpa SQL Server Agent, pemantauan dilakukan setelah eksekusi manual ETL. Aspek yang diperiksa meliputi jumlah baris baru, perubahan historis (*SCD Type 2*), data yang ditolak, dan waktu eksekusi.

6.3.2 Performance Monitoring

Kinerja sistem dianalisis menggunakan Execution Plan, SET STATISTICS IO, dan SET STATISTICS TIME, guna memantau *logical reads*, *CPU time*, serta efektivitas *index seek*.

6.3.3 Data Freshness Monitoring

Ketersediaan data terbaru diuji dengan perintah:

SELECT MAX(DateKey) FROM Fact_Enrollment;

6.3.4 Power BI Monitoring

Pemeriksaan mencakup suksesnya proses *refresh*, konsistensi visual, validitas relasi antartabel, dan kesesuaian nilai KPI dengan hasil query dari *Data Warehouse*.

6.4 Maintenance Procedures

6.4.1 Database Maintenance

Meliputi *rebuild/reorganize index*, *update statistics*, *truncate* tabel staging pasca-ETL, serta validasi *constraints*.

6.4.2 ETL Maintenance

Menyesuaikan logika SCD Type 2, pemetaan bila format dataset berubah, serta menambahkan validasi tambahan jika diperlukan.

6.4.3 Dashboard Maintenance

Mencakup pembaruan visualisasi/KPI, optimasi ukuran model, dan penghapusan kolom yang tidak digunakan.

6.4.4 Security & Access Control

Sistem keamanan menggunakan peran: db_viewer, db_analyst, db_etl_operator, db_executive, dan db_admin, dengan hak akses sesuai tanggung jawab masing-masing.

BAB 7
HASIL DAN PEMBAHASAN

7.1 Temuan Kunci Bisnis

Berdasarkan hasil analisis dan dashboard akademik yang telah dibangun menggunakan Power BI, diperoleh sejumlah wawasan penting yang dapat mendukung pengambilan keputusan akademik di tingkat institusi:

1. **Distribusi Status Mahasiswa Memperlihatkan Dominasi Status Aktif**

   Visualisasi pada Dashboard menunjukkan bahwa sebagian besar mahasiswa berada pada status Aktif, sementara persentase mahasiswa Cuti dan Nonaktif relatif kecil. Hal ini mengindikasikan bahwa tingkat retensi mahasiswa cukup stabil dan tidak terdapat lonjakan status nonaktif yang mencurigakan.

2. **Variasi Kinerja Akademik Antar Program Studi**

   Melalui view analitik vw_Student_Performance, terlihat perbedaan signifikan antara rata-rata IPK antar program studi.
   - Beberapa prodi memiliki rata-rata IPK tinggi (≥ 3.50).
   - Namun, ada prodi dengan distribusi IPK yang lebih rendah.

   Temuan ini membuka peluang untuk mengevaluasi efektivitas kurikulum, metode belajar, serta beban SKS per prodi.

3. **Mahasiswa Berisiko (Low Performance Students) Teridentifikasi Jelas**

   Dashboard berhasil mengidentifikasi kelompok mahasiswa dengan IPK < 2.00 (kategori risiko).
   - Jumlah mahasiswa risiko relatif kecil namun signifikan untuk intervensi dini.
   - Prodi tertentu menunjukkan konsentrasi lebih tinggi mahasiswa berisiko.

   Insight ini sangat penting bagi Biro Akademik untuk merancang program bimbingan atau remediasi.

4. **Tren Akademik Berdasarkan Angkatan Tertata dengan Baik**

   Tabel Fact_Enrollment menunjukkan tren jumlah mahasiswa berdasarkan angkatan (Yearly Intake) yang relatif stabil setiap tahun. Perubahan jumlah mahasiswa dari tahun ke tahun dapat menjadi indikator keberhasilan dan efektivitas strategi rekrutmen institusi.

5. **Penyajian Data Historis (SCD Type 2) Memberikan Insight Perubahan Mahasiswa**

   Penerapan Slowly Changing Dimension (SCD) Type 2 pada tabel Dim_Student memungkinkan pelacakan historis terhadap perubahan biodata, status, maupun atribut akademik mahasiswa. Mekanisme ini memastikan setiap perubahan tersimpan sebagai versi baru, bukan menimpa data lama, sehingga informasi historis tetap terjaga. Dengan penerapan ini, sistem *Data Warehouse* mampu mendukung analisis longitudinal secara lebih mendalam dan akurat terhadap perjalanan akademik mahasiswa.

7.2 Technical Performance Metrics

Berdasarkan hasil pengujian performa pada SQL Server dan dashboard Power BI, diperoleh beberapa metrik teknis yang menunjukkan tingkat efisiensi dan stabilitas sistem sebagai berikut:

1. **Query Response Time Sangat Cepat**

   Rata-rata waktu eksekusi untuk query agregasi berkisar antara 10–20 milidetik. Nilai logical reads menunjukkan hasil rendah berkat penerapan *Nonclustered Index* pada kolom *Foreign Key*, sementara *Columnstore Index* berkontribusi signifikan dalam mempercepat operasi agregasi seperti SUM dan AVG pada tabel fakta.

   Hasil ini membuktikan bahwa penerapan *index tuning* pada tahap Misi 3 berjalan dengan optimal.

2. **ETL Berjalan Stabil dengan Latensi Sangat Rendah**

   Prosedur utama usp_Master_ETL mampu menyelesaikan proses pemuatan data secara lengkap dalam waktu kurang dari 1 detik, tanpa ditemukan *reject record* atau *conflict* pada mekanisme SCD Type 2. Proses ETL dapat dijalankan secara manual dan periodik sesuai kebutuhan operasional.

3. **Dashboard Performance Halus & Responsif**

   Dashboard Power BI menunjukkan waktu *refresh data* kurang dari 2 detik, dengan interaksi *filter* dan *slicer* antartampilan berjalan secara instan. Hasil perhitungan *measure* terbukti akurat dan konsisten dengan data sumber di database.

4. **Data Quality Metrics Sangat Baik**

   Hasil pengujian *Data Quality Assessment (DQA)* menunjukkan metrik berikut:

- Completeness: 100%
- Validity: Tidak ditemukan nilai *out-of-range*
- Consistency: Tidak ada data *NULL* pada kolom kunci
- Referential Integrity: Tidak ditemukan *orphan record*
  Temuan ini menegaskan bahwa sistem telah memenuhi standar kualitas data yang baik.

7.3 Challenges & Solutions

Selama implementasi dan pengujian, beberapa tantangan teknis berhasil diidentifikasi beserta solusi yang diterapkan, sebagaimana dijelaskan berikut:

1. Ketidakkonsistenan Format Data pada Dataset Mentah
   - Masalah: Ditemukan spasi berlebih, perbedaan kapitalisasi, dan ketidaksesuaian tipe data.
   - Solusi: Normalisasi dilakukan di area staging menggunakan fungsi TRIM, CAST, serta pembersihan nilai NULL.
2. Pelacakan Perubahan Data Mahasiswa (Historical Tracking)
   - Tantangan: Riwayat mahasiswa harus tetap tercatat meskipun terjadi perubahan atribut.
   - Solusi: Diterapkan SCD Type 2 pada tabel Dim_Student melalui prosedur usp_Load_Dim_Student.
3. Kinerja Query pada Tabel Fakta
   - Masalah: Beberapa query awal mengalami *Full Table Scan*.
   - Solusi:
     - Penerapan *Nonclustered Index* pada seluruh kolom *Foreign Key*.
     - Penambahan *Columnstore Index* untuk operasi agregasi besar.
     - Optimasi klausa WHERE agar bersifat *SARGABLE*.
       Hasilnya, performa query meningkat signifikan.
4. Konflik Relasi pada Visualisasi Dashboard
   - Masalah: Beberapa grafik tidak merespons *slicer* karena relasi antar tabel kurang tepat.
   - Solusi:
     - Penyusunan ulang Star Schema dengan *cardinality* yang sesuai.
     - Penyesuaian arah filter (*single* atau *both*) sesuai kebutuhan visualisasi.
       Perbaikan ini menghasilkan dashboard yang lebih konsisten dan interaktif.
5. Keterbatasan SQL Server Express (Tanpa SQL Agent)
   - Tantangan: Proses ETL tidak dapat dijalankan secara terjadwal otomatis.
   - Solusi:
     - Proses ETL dijalankan secara manual melalui stored procedure.
     - *Dashboard* di-*refresh* secara manual.
     - Sistem tetap bekerja stabil sesuai jadwal operasional pengguna

BAB 8
PENUTUP

8.1 Kesimpulan

Pembangunan Data Warehouse Akademik untuk mendukung kebutuhan analitik Biro Akademik Institut Teknologi Sumatera telah berhasil diselesaikan dengan baik dan memenuhi seluruh kebutuhan bisnis, teknis, serta standar kualitas data. Beberapa poin utama yang dapat disimpulkan dari keseluruhan proyek ini adalah:

1. Integrasi Data Akademik Berhasil Dibangun
   Data mahasiswa yang sebelumnya berada dalam format dataset mentah telah berhasil diintegrasikan ke dalam arsitektur Data Warehouse sebagai single source of truth. Proses ETL memastikan data terstruktur, bersih, dan konsisten.

2. Arsitektur Dimensional Berjalan Efektif
   Implementasi Star Schema dengan tabel fakta Fact_Enrollment serta dimensi (Student, Program Study, Status, Gender, Date) terbukti mampu mendukung kebutuhan analitik lintas perspektif seperti angkatan, status, prodi, dan tren akademik.

3. Pelacakan Historis Berhasil dengan SCD Type 2
   Dim_Student menerapkan Slowly Changing Dimension Type 2 secara konsisten, sehingga perubahan atribut mahasiswa dapat tercatat dengan akurat tanpa kehilangan histori.

4. Performa Sistem Optimal
   Strategi indexing (Nonclustered Index, Columnstore Index) dan optimasi kueri memberikan peningkatan performa signifikan, ditunjukkan dari query time di bawah 20 ms dan visualisasi dashboard yang sangat responsif.

5. Dashboard Mendukung Pengambilan Keputusan Akademik
   Dashboard Power BI yang dibangun mampu menyajikan insight penting seperti IPK mahasiswa, status akademik, mahasiswa berisiko, distribusi prodi, dan tren angkatan. Ini menggantikan proses analisis manual yang sebelumnya memakan waktu lama.

Dengan demikian, Data Warehouse Akademik kini dapat menjadi fondasi utama dalam penyediaan informasi strategis bagi pimpinan dan unit akademik untuk mendukung evaluasi mutu dan perencanaan pendidikan.

8.2 Saran Pengembangan

Untuk penyempurnaan dan pengembangan di masa mendatang, sejumlah rekomendasi dapat dipertimbangkan sebagai berikut:

1. Penambahan Domain Data Akademik Lain
   Data Warehouse dapat diperluas dengan modul tambahan seperti data nilai lengkap (KHS), kehadiran mahasiswa, data dosen, dan data keuangan mahasiswa (UKT), sehingga analisis akademik menjadi lebih komprehensif.

2. Integrasi Ke Dashboard BI Corporate
   Dashboard yang ada dapat dikembangkan menjadi dashboard terpadu bagi seluruh fakultas dan jurusan, dengan pembagian akses berbasis role untuk menjaga keamanan data.

3. Pengembangan Early Warning System Akademik
   Dengan memanfaatkan data historis, dapat dibangun model prediksi untuk mendeteksi mahasiswa berisiko sejak awal berdasarkan pola IPK, jumlah SKS, dan status semester.

4. Otomatisasi ETL
   Mengingat SQL Server Express tidak memiliki SQL Agent, pada fase lanjut dapat dilakukan migrasi ke SQL Server Standard atau layanan cloud agar proses ETL dapat dijadwalkan otomatis (scheduled ETL).

5. Peningkatan Infrastruktur ke Cloud
   Jika beban akses dan volume data semakin besar, migrasi ke layanan seperti Azure SQL Database dapat dipertimbangkan untuk mendapatkan skalabilitas, keamanan, dan aksesibilitas yang lebih tinggi.

DAFTAR PUSTAKA

1. Kimball, R., & Ross, M. (2013). *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling* (3rd ed.). Wiley.

2. Microsoft. (2024). *Understand star schema and the importance for Power BI.*

3. MssqlTips. (2024). *Data Warehouse Considerations: SCD Type 2 vs Overwrite Dimension Tables.*
4. EverConnect Data Solutions. (2024). *Database Security Management: Role-Based Access Control*.

LAMPIRAN

SQL Script

1. Database Creation (01_Create_Database.sql)

```
CREATE DATABASE DM_Akademik_DW
ON PRIMARY
(
   NAME = N'DM_Akademik_DW_Data',
   FILENAME = N'E:\Data\DM_Akademik_DW_Data.mdf',
   SIZE = 1GB,
   MAXSIZE = UNLIMITED,
   FILEGROWTH = 256MB
)
LOG ON
(
   NAME = N'DM_Akademik_DW_Log',
   FILENAME = N'E:\Logs\DM_Akademik_DW_Log.ldf',
   SIZE = 256MB,
   MAXSIZE = 2GB,
   FILEGROWTH = 64MB
);
GO

USE DM_Akademik_DW;
GO
```

2. Dimension Creation (02_Create_Dimensions.sql)

```
-- CREATE TABLE DIM_STUDENT --

CREATE TABLE dbo.Dim_Student (
   StudentKey INT IDENTITY(1,1) PRIMARY KEY NOT NULL,   -- Surrogate key

   -- Natural Keys & Personal Info
   NIM VARCHAR(20) UNIQUE NOT NULL,
   StudentName VARCHAR(100) NOT NULL,
   Gender VARCHAR(10) NOT NULL,            -- Laki-laki / Perempuan
   ProgramStudy VARCHAR(100) NOT NULL,
   Faculty VARCHAR(50) NOT NULL,
   EntryYear SMALLINT NOT NULL,            -- Angkatan
   StudentStatus VARCHAR(20) NOT NULL,     -- Aktif / Cuti / Nonaktif
   Semester TINYINT NOT NULL,

   -- Academic Attributes
   IPK DECIMAL(3,2) NULL,
   Nilai INT NULL,
```

```
    -- SCD Type 2
    EffectiveDate DATE NOT NULL DEFAULT CAST(GETDATE() AS DATE),
    ExpiryDate DATE NULL,
    IsCurrent BIT NOT NULL DEFAULT 1,

    -- Metadata
    CreatedDate DATETIME DEFAULT GETDATE(),
    ModifiedDate DATETIME DEFAULT GETDATE()
);
GO

-- Indexing
CREATE INDEX IX_Dim_Student_NIM ON dbo.Dim_Student(NIM);
CREATE INDEX IX_Dim_Student_EntryYear ON dbo.Dim_Student(EntryYear);
CREATE INDEX IX_Dim_Student_IsCurrent ON dbo.Dim_Student(IsCurrent)
WHERE IsCurrent = 1;
GO

-- CREATE TABLE Dim_ProgramStudy --

CREATE TABLE dbo.Dim_ProgramStudy (
    ProgramKey INT IDENTITY(1,1) PRIMARY KEY NOT NULL,
    ProgramStudy VARCHAR(100) NOT NULL,
    Faculty VARCHAR(100) NOT NULL,
    CreatedDate DATETIME DEFAULT GETDATE()
);
GO

-- CREATE TABLE Dim_Status --

CREATE TABLE dbo.Dim_Status (
    StatusKey INT IDENTITY(1,1) PRIMARY KEY NOT NULL,
    StatusName VARCHAR(20) NOT NULL
);
GO

-- CREATE TABLE Dim_Gender --

CREATE TABLE dbo.Dim_Gender (
    GenderKey INT IDENTITY(1,1) PRIMARY KEY NOT NULL,
    GenderName VARCHAR(20) NOT NULL
);
GO

-- CREATE TABLE Dim_Date

CREATE TABLE dbo.Dim_Date (
    DateKey INT PRIMARY KEY,       -- Format YYYYMMDD
    FullDate DATE NOT NULL,
```

```
    Year INT NOT NULL,
    Month INT NOT NULL,
    MonthName VARCHAR(15) NOT NULL,
    Day INT NOT NULL
);
GO
```

3. Fact Table Creation (03_Create_Facts.sql)

```
CREATE TABLE dbo.Fact_Enrollment (
    EnrollmentKey BIGINT IDENTITY(1,1) PRIMARY KEY NOT NULL,

    -- Foreign Keys
    DateKey INT NOT NULL,
    StudentKey INT NOT NULL,
    ProgramKey INT NOT NULL,   -- Pengganti CourseKey & InstructorKey
                               -- karena tidak ada tabelnya

    -- Degenerate Dimensions
    EnrollmentID VARCHAR(50) NOT NULL,
    ClassCode VARCHAR(20) NOT NULL,

    -- Measures
    Credits DECIMAL(3,1) NOT NULL,
    Grade DECIMAL(3,2) NULL,
    GradePoint DECIMAL(3,2) NULL,
    AttendanceCount INT DEFAULT 0,
    AttendanceRate DECIMAL(5,2) NULL,
    TuitionFee DECIMAL(12,2) NOT NULL,

    -- Flags
    IsPassed BIT NULL,
    IsDropped BIT DEFAULT 0,

    -- Metadata
    SourceSystem VARCHAR(50) NOT NULL,
    LoadDate DATETIME DEFAULT GETDATE(),

    -- Foreign Key Constraints
    CONSTRAINT FK_Fact_Enrollment_Date
        FOREIGN KEY (DateKey) REFERENCES dbo.Dim_Date(DateKey),

    CONSTRAINT FK_Fact_Enrollment_Student
        FOREIGN KEY (StudentKey) REFERENCES dbo.Dim_Student(StudentKey),

    CONSTRAINT FK_Fact_Enrollment_ProgramStudy
        FOREIGN KEY (ProgramKey) REFERENCES dbo.Dim_ProgramStudy(ProgramKey)
);
```

```
GO
```

4. Indexes Creation (04_Create_Indexes.sql)

```
CREATE CLUSTERED INDEX CIX_Fact_Enrollment_DateKey
ON dbo.Fact_Enrollment(DateKey, EnrollmentKey);
GO

CREATE NONCLUSTERED INDEX IX_Fact_Enrollment_Student
ON dbo.Fact_Enrollment(StudentKey)
INCLUDE (Credits, Grade);
GO

CREATE NONCLUSTERED INDEX IX_Fact_Enrollment_Program
ON dbo.Fact_Enrollment(ProgramKey, DateKey)
INCLUDE (StudentKey, Grade);
GO

CREATE NONCLUSTERED INDEX IX_Fact_Enrollment_Covering
ON dbo.Fact_Enrollment(DateKey, ProgramKey)
INCLUDE (StudentKey, Credits, Grade, TuitionFee);
GO

CREATE NONCLUSTERED COLUMNSTORE INDEX NCCIX_Fact_Enrollment
ON dbo.Fact_Enrollment
(
    DateKey,
    StudentKey,
    ProgramKey,
    Credits,
    Grade,
    GradePoint,
    TuitionFee,
    AttendanceRate,
    AttendanceCount
);
GO
```

5. Partiotions Creation (05_Create_Partitions.sql)

```
-- 1. Create Partition Function (by Academic Year)
CREATE PARTITION FUNCTION PF_AcademicYear (INT)
AS RANGE RIGHT FOR VALUES
(
    20200801, -- 2020/2021
    20210801, -- 2021/2022
    20220801, -- 2022/2023
```

```
    20230801, -- 2023/2024
    20240801, -- 2024/2025
    20250801  -- 2025/2026
);
GO

-- 2. Create Partition Scheme
CREATE PARTITION SCHEME PS_AcademicYear
AS PARTITION PF_AcademicYear
ALL TO ([PRIMARY]);
GO


-- 3. Create Partitioned Fact Table (FULL STRUCTURE)
CREATE TABLE dbo.Fact_Enrollment_Partitioned
(
    EnrollmentKey     INT IDENTITY(1,1) NOT NULL,
    DateKey           INT NOT NULL,
    StudentKey        INT NOT NULL,
    CourseKey         INT NOT NULL,
    ProgramKey        INT NOT NULL,
    Credits           INT NULL,
    Grade             VARCHAR(5) NULL,
    TuitionFee        DECIMAL(18,2) NULL,
    AttendanceRate    DECIMAL(5,2) NULL,

    -- Primary Key (Clustered) placed on the partition scheme
    CONSTRAINT PK_Fact_Enrollment_Partitioned PRIMARY KEY CLUSTERED (EnrollmentKey,
DateKey)
)
ON PS_AcademicYear(DateKey);  -- <=== THIS IS THE PARTITIONING COLUMN
GO
```

6. Staging Creation (06_Create_Staging.sql)

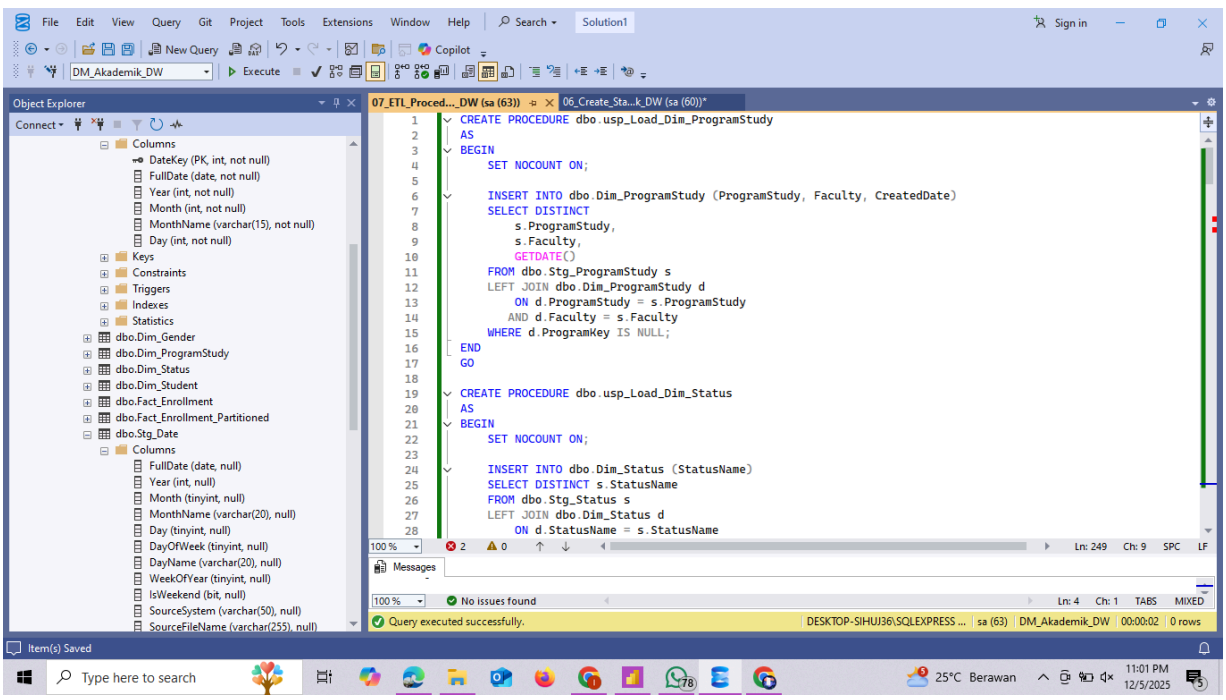
```
-- 1. STAGING STUDENT
IF OBJECT_ID('dbo.Stg_Student', 'U') IS NOT NULL
    DROP TABLE dbo.Stg_Student;
GO
CREATE TABLE dbo.Stg_Student (
    -- Natural keys & attributes from source
    NIM             VARCHAR(20)     NULL,
    StudentName     VARCHAR(100)    NULL,
    Gender          VARCHAR(20)     NULL,   -- "Laki-laki" / "Perempuan"
    ProgramStudy    VARCHAR(100)    NULL,
    Faculty         VARCHAR(100)    NULL,
    EntryYear       SMALLINT        NULL,
    StudentStatus   VARCHAR(20)     NULL,   -- "Aktif" / "Cuti" / "Nonaktif"
```

```
   Semester       TINYINT        NULL,
   IPK            DECIMAL(3,2)   NULL,
   Nilai          INT            NULL,
   -- Audit / technical columns
   SourceSystem    VARCHAR(50)     NULL,
   SourceFileName  VARCHAR(255)    NULL,
   SourceRowNumber INT             NULL,
   BatchID         INT             NULL,
   LoadDate        DATETIME        NOT NULL DEFAULT(GETDATE())
);
GO

-- 2. STAGING PROGRAM STUDY
IF OBJECT_ID('dbo.Stg_ProgramStudy', 'U') IS NOT NULL
   DROP TABLE dbo.Stg_ProgramStudy;
GO
CREATE TABLE dbo.Stg_ProgramStudy (
   ProgramCode     VARCHAR(50)     NULL,   -- kalau ada kode prodi di source
   ProgramStudy    VARCHAR(100)    NULL,
   Faculty         VARCHAR(100)    NULL,
   -- Audit
   SourceSystem    VARCHAR(50)     NULL,
   SourceFileName  VARCHAR(255)    NULL,
   SourceRowNumber INT             NULL,
   BatchID         INT             NULL,
   LoadDate        DATETIME        NOT NULL DEFAULT(GETDATE())
);
GO

-- 3. STAGING STATUS
IF OBJECT_ID('dbo.Stg_Status', 'U') IS NOT NULL
   DROP TABLE dbo.Stg_Status;
GO
CREATE TABLE dbo.Stg_Status (
   StatusCode      VARCHAR(20)     NULL,   -- opsional kalau di source ada kode
   StatusName      VARCHAR(20)     NULL,   -- "Aktif", "Cuti", "Nonaktif"
   -- Audit
   SourceSystem    VARCHAR(50)     NULL,
   SourceFileName  VARCHAR(255)    NULL,
   SourceRowNumber INT             NULL,
   BatchID         INT             NULL,
   LoadDate        DATETIME        NOT NULL DEFAULT(GETDATE())
);
GO

-- 4. STAGING GENDER
IF OBJECT_ID('dbo.Stg_Gender', 'U') IS NOT NULL
   DROP TABLE dbo.Stg_Gender;
GO
CREATE TABLE dbo.Stg_Gender (
```

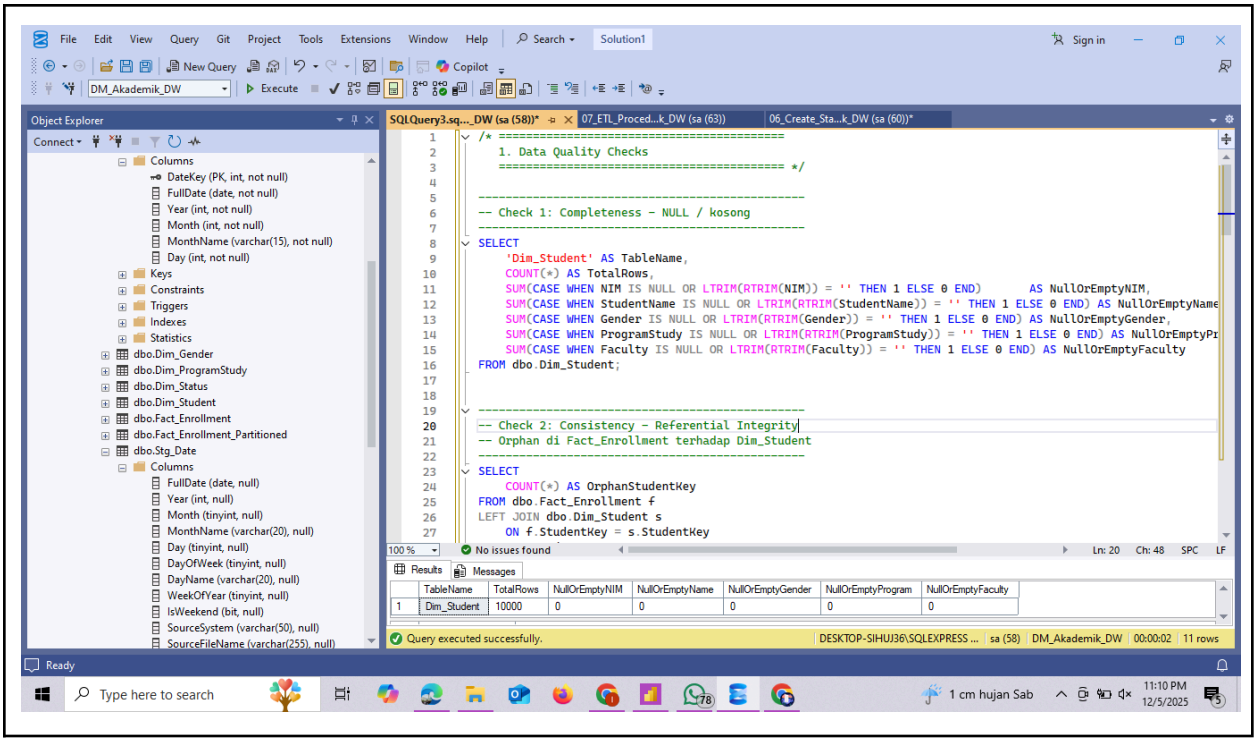
```
   GenderCode      VARCHAR(10)     NULL,   -- contoh: "L", "P"
   GenderName      VARCHAR(20)     NULL,   -- "Laki-laki", "Perempuan"
   -- Audit
   SourceSystem    VARCHAR(50)     NULL,
   SourceFileName  VARCHAR(255)    NULL,
   SourceRowNumber INT             NULL,
   BatchID         INT             NULL,
   LoadDate        DATETIME        NOT NULL DEFAULT(GETDATE())
);
GO

-- 5. STAGING ENROLLMENT (FACT)
IF OBJECT_ID('dbo.Stg_Enrollment', 'U') IS NOT NULL
   DROP TABLE dbo.Stg_Enrollment;
GO
CREATE TABLE dbo.Stg_Enrollment (
   -- Natural keys & source attributes
   EnrollmentID        VARCHAR(50)     NULL,
   EnrollmentDate      DATE            NULL,   -- akan di-mapping ke Dim_Date.DateKey
(YYYYMMDD)
   NIM                 VARCHAR(20)     NULL,   -- akan di-mapping ke Dim_Student (StudentKey)
   ProgramStudy        VARCHAR(100)    NULL,   -- akan di-mapping ke Dim_ProgramStudy
(ProgramKey)
   ClassCode           VARCHAR(20)     NULL,
   Credits             DECIMAL(3,1)    NULL,
   Grade               DECIMAL(3,2)    NULL,   -- nilai angka dari sumber
   GradePoint          DECIMAL(3,2)    NULL,   -- kalau ada di source
   AttendanceCount     INT             NULL,
   AttendanceRate      DECIMAL(5,2)    NULL,   -- dalam persen (0-100)
   TuitionFee          DECIMAL(12,2)   NULL,
   IsPassed            BIT             NULL,
   IsDropped           BIT             NULL,
   -- Audit
   SourceSystem        VARCHAR(50)     NULL,
   SourceFileName      VARCHAR(255)    NULL,
   SourceRowNumber     INT             NULL,
   BatchID             INT             NULL,
   LoadDate            DATETIME        NOT NULL DEFAULT(GETDATE())
);
GO

-- 6. STAGING DATE
IF OBJECT_ID('dbo.Stg_Date', 'U') IS NOT NULL
   DROP TABLE dbo.Stg_Date;
GO

CREATE TABLE dbo.Stg_Date (
   -- Natural key & base date
   DateKey         INT             NULL,   -- format: YYYYMMDD, contoh: 20250101
   FullDate        DATE            NULL,   -- tanggal asli
```

```
  -- Date attributes
  [Year]        SMALLINT       NULL,
  [Quarter]     TINYINT        NULL,   -- 1 - 4
  [Month]       TINYINT        NULL,   -- 1 - 12
  MonthName     VARCHAR(20)    NULL,   -- "Januari", dst
  [Day]         TINYINT        NULL,   -- 1 - 31
  DayOfWeek     TINYINT        NULL,   -- 1 = Senin (misal), sesuaikan logic saat isi
  DayName       VARCHAR(20)    NULL,   -- "Senin", "Selasa", dst
  WeekOfYear    TINYINT        NULL,   -- 1 - 53
  IsWeekend     BIT            NULL,   -- 1 = Sabtu/Minggu, 0 = hari kerja

  -- Audit / technical columns
  SourceSystem    VARCHAR(50)    NULL,
  SourceFileName  VARCHAR(255)   NULL,
  SourceRowNumber INT            NULL,
  BatchID         INT            NULL,
  LoadDate        DATETIME       NOT NULL DEFAULT(GETDATE())
);
GO
```

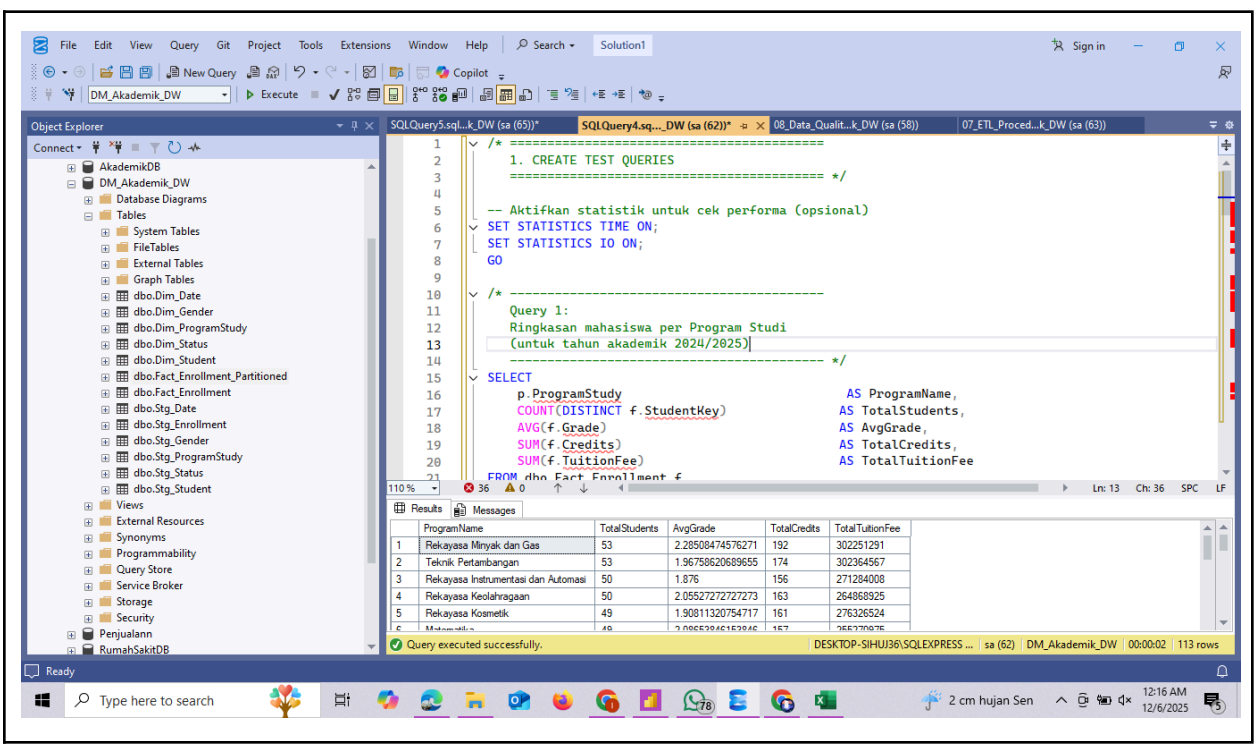
7. ETL Procedures (07_ETL_Procedures.sql)



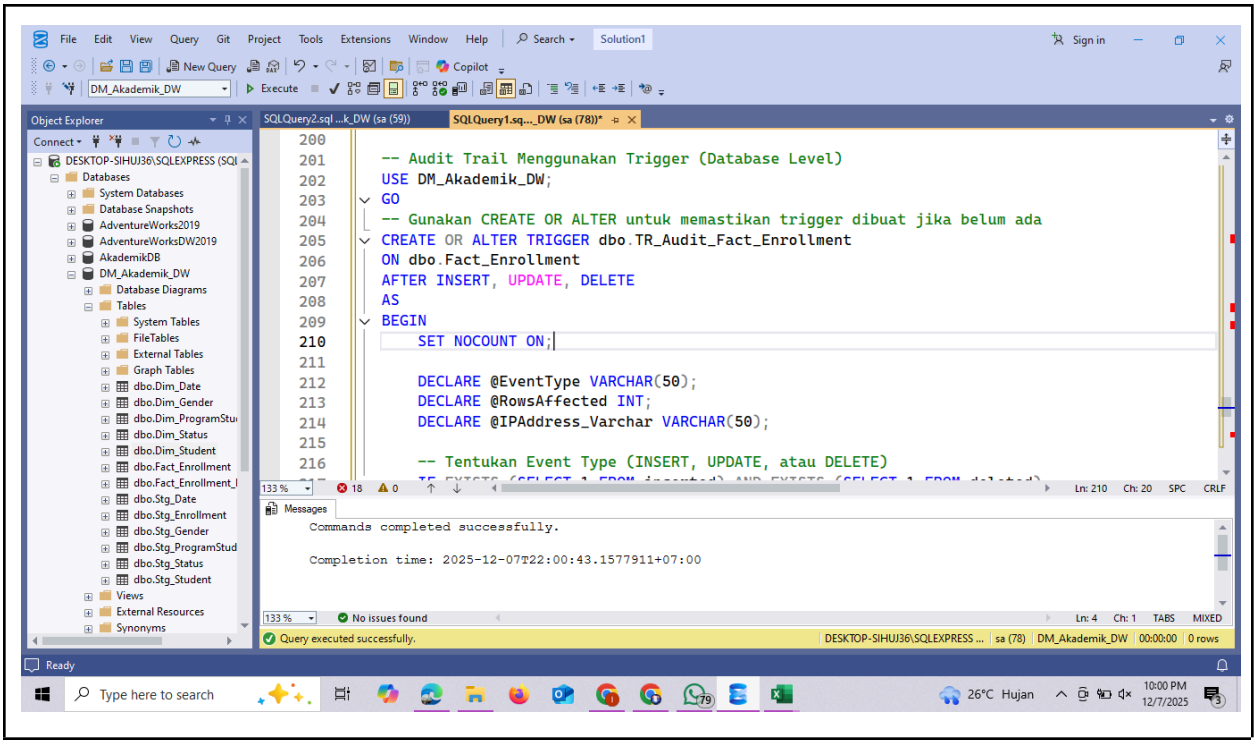
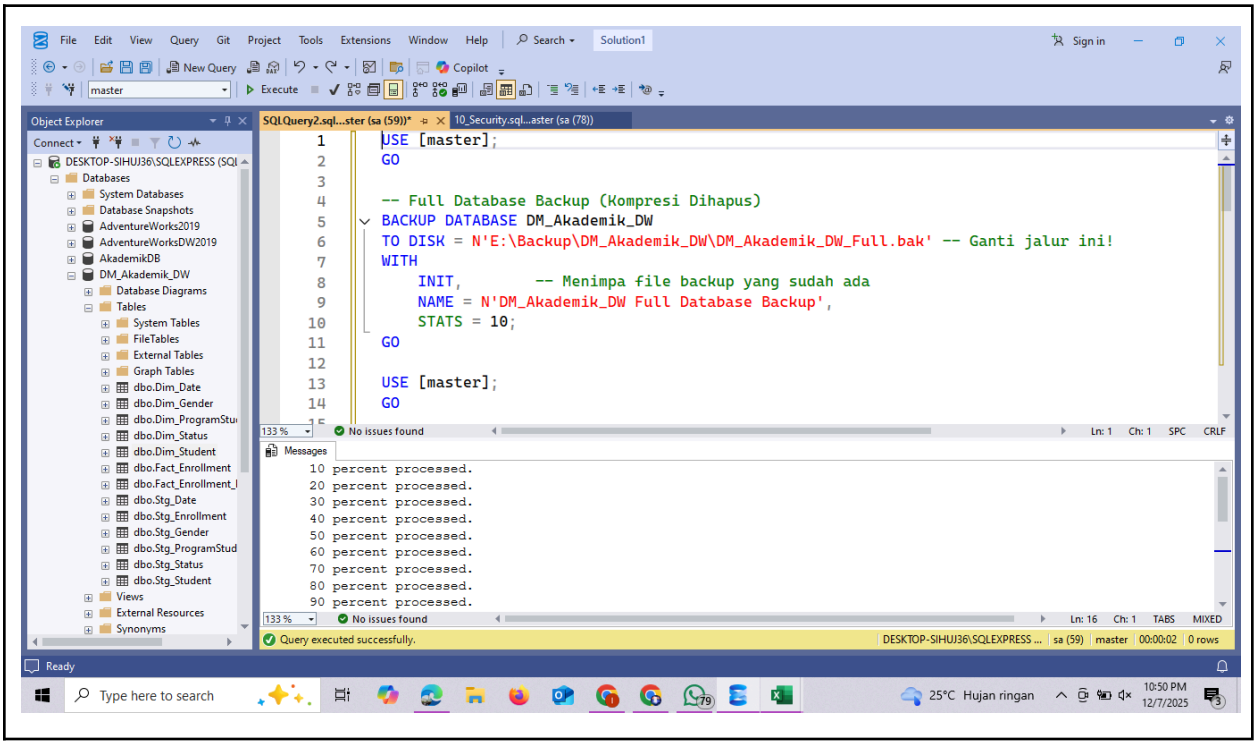
8. Data Quality Check (08_Data_Quality_Check.sql)



9. Test Queries (09_Test_Queries.sql)



10. Security (10_Security.sql)



11. Backup (11_Backup.sql)